# POLYAS 3.0 Verifiable E-Voting Protocol

Version 2.2

October 9, 2026

Tomasz Truderung

POLYAS GmbH

## Contents

**1 Introduction** **7**

**2 Overview of the Protocol and Security Properties** **9**

2.1 Overview of the Protocol . . . 9

2.2 Ballot Privacy . . . 11

2.3 Everlasting Privacy . . . 11

2.4 Universal verifiability . . . 12

2.5 Protection against ballot stuffing . . . 13

2.6 Individual (cast-as-intended) verifiability . . . 14

**3 Overview of the Verification Tasks** **14**

**A Cryptographic Modules** **19**

A.1 Auxiliary Algorithms . . . 19

A.1.1 KDF . . . 19

A.1.2 Hashing into $\mathbb{Z}_q$ . . . 21

A.1.3 Encoding Plaintexts as Numbers . . . 23

A.2 Used ElGamal Group . . . 23

A.2.1 Elliptic curve encoding . . . 25
A.2.2 Select Generator Verifiably . . . 26
A.3 Everlasting Privacy . . . 27
A.4 Credential Generation . . . 29
A.5 Ballot Creation and Validation . . . 31
A.5.1 Public labels . . . 31
A.5.2 Ballot encoding, encryption, and casting . . . 32
A.5.3 Server-side ballot validation . . . 34
A.5.4 The verification task . . . 34
A.6 Verifiable Shuffle . . . 36
A.7 Key Generation . . . 45
A.8 Verifiable Decryption . . . 47
A.9 Final Ballot Tallying . . . 50

**B Cast-as-Intended with a Second Device** **53**
B.1 High level description of the process and information flow . . . 53
B.2 An Optimisation for Long Ballots . . . 54
B.3 The Setup . . . 55
B.4 Content of the QR code . . . 55
B.5 Login request and response . . . 56
B.6 Checking the initial response . . . 57
B.7 Challenge and the Final Message . . . 58
B.8 Decoding and displaying the voter's choice . . . 60
B.9 The Receipt . . . 62
B.10 Second Device Protocol: Example Transcript . . . 62
B.10.1 The content of the QR code . . . 62
B.10.2 The election status response . . . 62
B.10.3 The login request . . . 63
B.10.4 The login response . . . 63
B.10.5 The challenge request . . . 66

B.10.6 The challenge response: . . . 66

**C Additional Verification Tasks** **66**

C.1 Second Device Public Parameters . . . 66

C.2 Receipts . . . 68

**D Format of the Verification Data** **71**

**E Bulletin Boards** **72**

E.1 External boards . . . 73

E.1.1 Triggers . . . 73

E.1.2 External board registry . . . 73

E.1.3 External board ballot-box . . . 76

E.1.4 External board revocations . . . 78

E.2 Sub-component ballot . . . 78

E.2.1 Board ballot-flagged . . . 78

E.2.2 Board ballot-filtered-out . . . 81

E.3 Sub-component BBox . . . 82

E.3.1 Secret BBox-ballotbox-key-secret-CP . . . 82

E.3.2 Board BBox-ballotbox-key-CP . . . 82

E.4 Sub-component decryption . . . 83

E.4.1 Board decryption-decrypt-Polyas . . . 83

E.5 Sub-component final-guard . . . 84

E.5.1 Verification Component final-guard-Polyas . . . 84

E.6 Sub-component keygen . . . 85

E.6.1 Secret keygen-secretKey-Polyas . . . 85

E.6.2 Board keygen-electionKey-Polyas . . . 85

E.7 Sub-component mixing . . . 86

E.7.1 Board mixing-input-packets . . . 86

E.7.2 Board mixing-mix-Polyas . . . 87

E.8 Sub-component pre-decryption-guard . . . 89

E.8.1 Verification Component pre-decryption-guard-Polyas . . . . . 89

**F Data Types** **90**

F.1 Annotated<B> . . . . . 90
F.2 Ballot<GroupElem> . . . . . 90
F.3 BallotEncodingSpec . . . . . 91
F.4 BallotEpEntry<GroupElement> . . . . . 91
F.5 BallotEpPackage<GroupElement> . . . . . 92
F.6 BallotId . . . . . 92
F.7 BallotSpec . . . . . 92
F.8 CandidateListSpec . . . . . 94
F.9 CandidateSpec . . . . . 96
F.10 Ciphertext<GroupElement> . . . . . 97
F.11 ColumnProperties . . . . . 98
F.12 Content<T> . . . . . 98
F.13 Custom . . . . . 98
F.14 DecryptionZKP.Proof<GroupElem> . . . . . 98
F.15 DerivedCandidateVotesSpec . . . . . 99
F.16 DerivedCandidateVotesSpec.Variant . . . . . 99
F.17 DerivedListVotesSpec . . . . . 99
F.18 DerivedListVotesSpec.Variant . . . . . 99
F.19 DlogNIZKP.Proof . . . . . 100
F.20 ECElement . . . . . 100
F.21 EpRegistry<GroupElement> . . . . . 100
F.22 EpVoter<GroupElement> . . . . . 101
F.23 EqlogNIZKP.Proof . . . . . 101
F.24 I18n<T> . . . . . 102
F.25 Language . . . . . 102
F.26 Message . . . . . 102
F.27 MessageWithProof<G> . . . . . 102

F.28 MessageWithProofPacket<GroupElem> . . . 103
F.29 MixPacket<GroupElem> . . . 103
F.30 MultiCiphertext<GroupElement> . . . 104
F.31 Packet<A> . . . 104
F.32 PublicKeyWithZKP<GroupElem> . . . 104
F.33 PublicVoterId . . . 105
F.34 RevocationToken . . . 105
F.35 RichText . . . 105
F.36 RichTextDocument . . . 106
F.37 ShuffleZKProof<GroupElement> . . . 106
F.38 SigningKey . . . 107
F.39 SigningKeyPair . . . 107
F.40 Sparse . . . 107
F.41 Standard . . . 107
F.42 Status . . . 108
F.43 Text . . . 108
F.44 Type . . . 108
F.45 VerificationKey . . . 108
F.46 ZKPs . . . 108
F.47 ZKPt<GroupElement> . . . 109

# List of Algorithms

1 Key derivation function . . . 19
2 Numbers from seed . . . 20
3 Numbers from seed (range) . . . 21
4 Uniform Hash . . . 22
5 Encoding a message as a multi-plaintext . . . 24
6 Decoding a message from a multi-plaintext . . . 24
7 Independent generators for EC groups of prime order . . . 26

8 Verification of a ballot . . . . . 35
9 Interactive ZK-Proof of Extended Shuffle . . . . . 38
10 ZK-Proof of Shuffle Generation . . . . . 40
11 Verification of a ZK-proof of Shuffle . . . . . 42
12 Verification of the public election key with the ZK-proof . . . . . 46
13 Verification of ballot decryption . . . . . 49
14 Ballot as normalized bytestring . . . . . 59
15 Final ZKP validation . . . . . 61

# 1 Introduction

This document describes a variant of *POLYAS 3.0 E-Voting System* offering universal verifiability, individual (cast-as-intended) verifiability, and (everlasting) ballot privacy. We outline the structure of the protocol and the intended security properties. We also describe the tasks assigned to the Election Board in order to support some security aspects (such as universal verifiability). Further details, including in particular a detailed specification of the verification procedure, are provided in appendices.

The proposed system has the potential to provide a whole range of state-of-the-art security features. Some aspects of these security features require, however, participation of additional independent parties, according to the principle of division of roles; the role of such independent parties can be carried out by, for instance, entities designated by the Election Board or an organisation representing voters or candidates. While participation of such independent parties involves some additional organisational effort, this effort can be scaled depending on the required security level.

Our system provides the following security features safeguarding the integrity of the election process (see Section 3 for more details):

- **Universally verifiable tallying**, based on verifiable mixing and verifiable decryption, using state-of-the art cryptographic techniques, such as zero-knowledge proofs.
  The corresponding verification tools (implemented based on the specification provided in this document) can be used to verify correctness of the tallying process. A reference implementation of the verification procedure is provided by POLYAS (including the source code).
- **Eligibility verifiability.** Voters' credentials are generated by an entity designated by the Election Board. The credentials are then distributed to the voters using, for instance, a secure printing facility. This mechanism provides a measure against so-called *ballot stuffing*, that is unauthorized injection of additional ballots to the digital ballot box by the Election Provider (POLYAS).
  POLYAS provides a tool for credential generation, including the source code. Such a tool can be also independently implemented (by the Election Board or a third party).
- **Individual/cast-as-intended verifiability** based on the use of a second device by the voter.

The three above features combined provide so-called *end-to-end verifiability*. These verifiability guarantees are complemented by the following privacy guarantees:

- **Ballot privacy**, preventing any party from being able to associate an individual voter with the choice they have made. This property is based on end-to-end encryption, where a ballot is encrypted on the client side (in the voter's browser) and only decrypted during the tallying phase after cryptographic shuffling (which detaches ballots from the voter identities).
  In the standard configuration, POLYAS manages the private election key and is, therefore, trusted for ballot privacy. In an extended configuration (not described in this document), the private election key can be shared by some number of independent parties (trustees), which distributes the trust amongst these authorities in a threshold manner (participation of some predefined number of trustees, for example two or three, are needed to decrypt ballots).
- **Everlasting privacy**, preventing any external party from being able to associate an individual voter with the choice they have made even if (in the future) the used cryptographic primitives turn out to be broken (due to, for instance, advances in quantum computing). This is achieved by not storing the association between voters and their encrypted ballots already when published in the digital ballot box. For this property we need to assume that some components of election process are honest and operate according to the specification (which is a typical assumption for everlasting privacy).

Note that everlasting privacy gives seemingly stronger guarantees than standard ballot privacy: while standard ballot privacy assumes that the encryption method can be trusted, everlasting privacy does not make this assumption. On the other hand, everlasting privacy makes stronger trust assumptions: more components of the election process need to be trusted to not misbehave for this property to hold. This means that everlasting privacy is not meant to replace the "standard" notion of ballot privacy, but rather to complement it, making different trade-offs with respect to security and trust.

In what follows, we describe the general structure of the current configuration of *POLYAS Core 3.0 Verifiable* and discuss its intended security properties. Then we provide an overview of the verification tasks, that is tasks to be implemented in order to build a verification tool for universal verifiability of the tallying process. In the appendices we provide details on the used cryptographic modules and the protocol structure. These details should enable any interested party to independently implement a verification procedure.

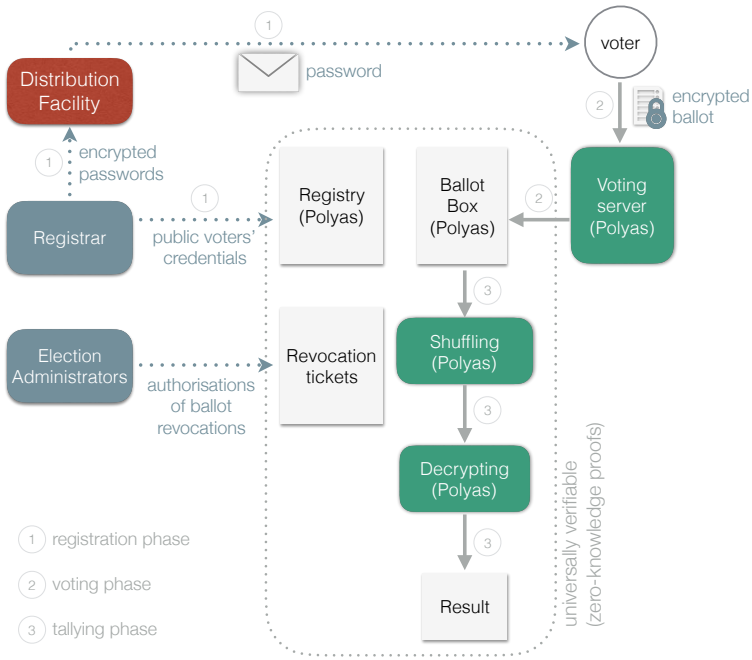



Figure 1: The structure of the system. Gray boxes represent bulletin boards. Ballot box is a bulletin board, where submitted ballots are published. Although not indicated on the picture, the intermediate results of shuffling (mixing) are also posted on bulletin boards and hence are subject to audit. In particular, verifiable shuffling and verifiable decryption produce zero-knowledge proofs of correctness of these operations.

# 2 Overview of the Protocol and Security Properties

In this section, we describe the structure of the protocol and provide more details on the security properties the system is aiming at, including the trust assumptions under which these security properties hold.

## 2.1 Overview of the Protocol

The protocol structure of the currently offered configuration is depicted on Figure 1.

The protocol involves the following roles:

- the *Election Board*
- the *Election Provider* (POLYAS),

- the *Registrar* and the *Distribution Facility*

Additionally, in the extended configuration with threshold description, some number of (independent) external *Trustees* take part in the protocol. These trustees participate in the generation of the secret election key (so that each holds their own share of this key) and, later, in the secure shuffling, and threshold decryption.

The roles of the Registrar, the Election Administrators, the Distribution Facility and (in the extended configuration) the trustees are, typically, assigned by the Election Board (the organisation carrying out the elections).

The system uses standard ElGamal-based cryptography with verifiable mixing (based on the construction of Wikstroem et al.) and verifiable decryption.

In the *registration phase* (Step 1 on Figure 1), the Registrar generates voters' private credentials and the corresponding public credentials (where private and public voters' credentials are in a cryptographic relation: roughly, the private credential serves as a signing key and the public credential is the corresponding verification key). The voters' private credentials are then distributed (via a Distribution Facility) to the voters (for instance by postal mail). The public credentials of all eligible voters are uploaded by the Registrar to the voting system (POLYAS) and published on the public registry board.

During the *voting phase* (Step 2), voters open the election web page (running the voting client code in the voters' browsers), authenticate themselves to the voting platform (using their private credentials and possibly additional authentication factors), and indicate their voting choices. The voting client creates an encrypted and signed ballot containing the selected choice (the ballot is signed with the voter's private key) and submits this ballot to the voting server which adds the ballot to the digital ballot box. Note that the ballot is encrypted and signed on the client side and hence the voter's choice does not leave the voter's computer unencrypted; also the voter's private key is never transferred to the server. At this point, the voter has an option to use a second device (such as a mobile phone or a tablet) to verify that the cast ballot contains in fact their intended choice (cast-as-intended verifiability).

The POLYAS Core 3 system can be configured to support so-called *ballot revocations*, a verifiable and transparent mechanism to revoke ballots of chosen voters. If the system is so configured, the Election Administrators can create and authorise, so called, *revocation tickets*. A revocation ticket includes a list of voters whose ballots are to be revoked. The *revocation policy* of the given election specifies the minimal required number of Election Administrators who should authorise a ticket. Ballots cast by the voters listed in the published revocation tickets do not take part in the tally (they are filtered out before the mixing step).

In the *tallying phase* (Step 3), the encrypted ballots from the ballot box (which are not revoked, as described above) are mixed and decrypted in a verifiable way (that is, zero-knowledge proofs of correct shuffle and correct decryption are produced), which means that the tallying process is fully verifiable. As explained later, the Election Board (as well as any other parties with the access to the verification data) can verify the proofs to make sure that tallying was carried out correctly.

We will now discuss the security features the system is designed to provide.

## 2.2 Ballot Privacy

In the variant of the POLYAS Core 3 system described on this document, POLYAS generates and maintains the private elections key and publishes the corresponding public election key (used to encrypt ballots). Therefore, POLYAS is trusted to handle and use the private election key properly (that is use this key exactly as specified by the protocol), in order to provide ballot privacy.

We also offer a variant of the system (not described in this document) using a threshold decryption scheme, where the private election key is shared between the election system and some number of independent (external) actors (trustees). These authorities participate in cryptographic shuffling and decryption. In this case, one does not need to trust any individual party for ballot secrecy.

## 2.3 Everlasting Privacy

Additionally to the discussed above ballot privacy, the system offers a stronger property of *everlasting privacy*. This property means that, under reasonable assumptions, the secrecy of each ballot is protected indefinitely, even if the cryptographic techniques used today become vulnerable or compromised at some point in the future.

The method is described in Section A.3. It is based on the following key points. Encrypted ballots stored in the digital ballot box do not include voter identifiers. In fact, the system removes any link between a stored ballot and the voter that might enable anyone *besides the voter* to associate them with their stored ballot. More specifically, the association between the given ballot and the voter identity can only be established using a dedicated secret known only to the voter (under the trust assumptions listed below).

*Trust assumptions.* The system provides everlasting privacy, based on the assumptions that (i) the Registrar honestly carries out the registration phase, (ii) the voting server honestly carries out the ballot cast procedure, (iii), if the voters uses a second device

application for the cast-as-intended verification,

The important practical benefit of the utilized method is that the packet of data for universal verifiability, which can be handed out to many external parties, will never reveal how individual voters voted, even if the used encryption method will be rendered insufficient due to crypto-analytical breakthroughs (such as advances in quantum computing).

## 2.4 Universal verifiability

As already mentioned, the complete tallying process (taking as its input the content of the ballot box, the registration board, and, if ballot revocations are supported by the election, the content of the revocation boards) is fully verifiable. It uses standard cryptographic methods to achieve this end, including fully verifiable shuffle and the decryption by the means of zero-knowledge proofs.

The universal verifiability mechanism discussed here allows one to check the integrity of the tallying process without any trust assumptions.

The process is organised as follows. At the end of the election, POLYAS provides the Election Board with the data packet containing the election result, along with the proof of correctness of the tallying process. More precisely, this data packet contains the content of all the public bulletin boards which include *the final content of the ballot box*, as well as the results of all the intermediate steps with appropriate zero-knowledge proofs. The Election Board (auditors) can then check the proofs (using a verification tool), in order to confirm that the tallying has been done correctly (and the election result is correct with respect to the included content of the ballot box).

The key bulletin boards relevant for the verification process are

- **the registry** containing, most importantly, the list of public credentials of eligible voters,
- **the ballot box** containing the final list of encrypted ballots to be tallied; the verification tool should check that all the ballots are well-formed and properly signed (using secret credentials corresponding to the public credentials published in the registry),
- **the revocation board**, present only if the election is configured to support revocations, containing revocation tokens.
- **the result of verifiable shuffling** with the zero-knowledge proofs of correct shuffle; the verification tool should check these proofs,
- **the decrypted ballots** along with the zero-knowledge proofs of correct decryption; the verification tool should check these proofs.

Note that the actual final tally (that is the association of the number of votes to individual candidates/choices) can be computed based on the content of the last bulletin board.

Detailed cryptographic and technical specification of the tallying process, the intended content and data format of all the public bulletin boards, and the corresponding verification procedure, are provided in the Appendix. Based on this specification, independent parties can implement verification tools (which can be used by the Election Board). POLYAS also provides a reference implementation of such a tool with the source code.

## 2.5 Protection against ballot stuffing

The ballot box is maintained and controlled by POLYAS which is supposed to publish there only encrypted ballots submitted by correctly authorized voters. To avoid hypothetical overuse of the control over the bulletin board (by adding additional illegitimate ballots), the system implements a protection mechanism based on the use of independently generated voter's credentials.

Such credentials are generated by an independent party called the *Registrar*: each eligible voter obtains their private credential, while all the corresponding public credentials are uploaded by the Registrar to the election registry board.

During the voting phase, the voter's private credential is used (together with other authentication factors), to authenticate the voter and then to sign the encrypted ballot (where the signing key is derived from the private credential). The signature is checked, first, by the voting server (to make sure that the ballot is authorized) and later by the verification procedure (carried out, for example, by the Election Board).

We note here the following properties of this process, which provide countermeasures against ballot stuffing:

- To create a valid ballot, the private credential of an eligible voter is needed. Adding a ballot which has not been signed with a valid private credential would be detected by the verification procedure.
- POLYAS does not have the knowledge of voter's private credentials and is therefore not able to fabricate ballots of voters who have not voted.
  This property holds under the assumption that the Registrar and the Distribution Facility do not collude with distribution/printing facility does not collude with POLYAS.

POLYAS provides an application, including the source code, for generating secret/public voters' credentials. Such an application can be also implemented independently.

See Section A.4 for the details of the password generation process.

*Trust assumptions.* The system provides protection against ballot stuffing, as defined above, under the following trust assumptions:

- The Registrar is honest (does not manipulate the software for credential generation in order to circumvent the cryptographic measures used by this software to protect these credentials).
- The Distribution Facility is honest (does not reveal voter's credentials to any third parties, in particular to the Election Provider).

### 2.6 Individual (cast-as-intended) verifiability

The Polyas Core 3 Verifiable e-voting system offers an option of the cast-as-intended verifiability, where voters can use a second-device, such as a mobile phone, to audit their ballots. This process is detailed in Section B. It is an instantiation of the method presented in [9, 8].

## 3 Overview of the Verification Tasks

In this section we describe architecture of the system in more detail, providing in particular the list of boards to be verified, and give an overview of the tasks of the verification procedure.

Bulletin boards of the described system are depicted on Figure 2 (where the revocation board is only present if the election is configured to support ballot revocations). The detailed list of all bulletin boards is given in Appendix E. Bulletin boards offer append-only storage facility. They provide the only way the data is transferred between the subcomponents of the election protocol, which makes the transfer of data (the protocol transcript) transparent and explicit. Content of these boards constitutes the input to the verification procedure.

The data published on the so-called, *external boards* (represented in orange) comes from the outside of the system: the registration data published on the registry board is generated externally by the credential generation tool, the revocation tickets published on the revocations board are created by election administrators, while encrypted ballots, published on the ballot-box board, are created on the client side during the voting casting process and published via the voting server. The content of the remaining (internal) boards is computed by the Polyas Core 3.0 back-end from the indicated input.

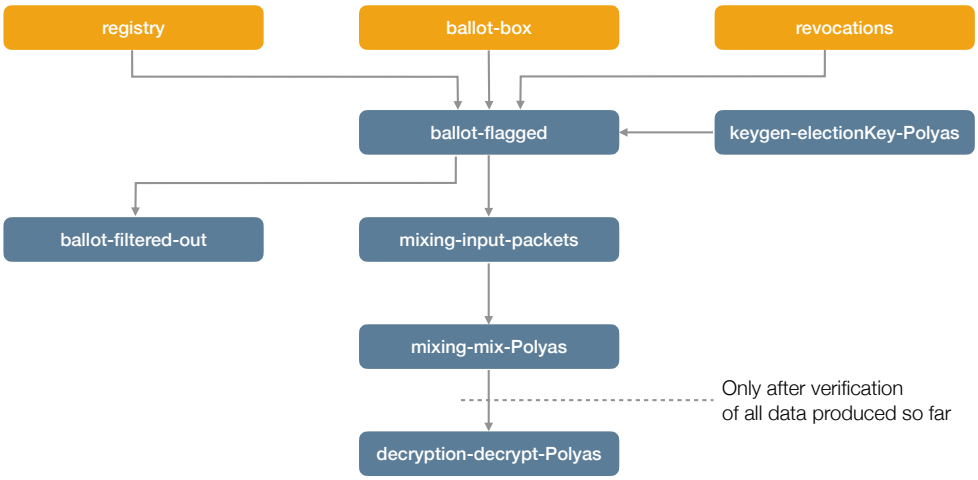



Figure 2: Bulletin boards of the system with indicated information flow between them. Orange boxes represent *external boards*; the content of all the remaining boards is computed (from the indicated input) by the Polyas Core 3.0 back-end. The figure presents the variant without key sharing and threshold decryption.

The goal of the verification process is to check that the content of all internal bulletin board has been computed correctly. Additionally one needs to make sure that the content of the external boards is as expected.

The cryptographic details of how the system works and how the verification of different boards should be implemented are given in the following sections. Here we provide an overview of the verification tasks.

**Cryptographic setup and auxiliary algorithms.** In order to implement the verification tool, one needs to first implement some auxiliary cryptographic algorithms. Such algorithms are described in Section A.1. This section includes, in particular, descriptions of a key derivation procedure (Algorithm 1) and an algorithm for a hashing into $\mathbb{Z}_q$ (Algorithm 4), which are building blocks of many verification algorithms.

The general cryptographic setting (the used ElGamal group and some algorithms for this group) is given in Section A.2. It includes, in particular, Algorithm 7 for deriving independent elements of the group in a reproducible (verifiable) way. Such independent elements (generators) are a necessary part of the proof of correct shuffle.

**Registry board.** The expected content of the registration board is described in Section A.4 (where the credential generation process is described) and in Section E.1.2, where the format of the registry data is specified. The verification task related to the content of this board is simply to make sure that the registration data is well-formed (as described in the mentioned section). The Election Board (who uploads the content of this board) needs to make sure that this board contains the intended content.

**Election key.** The election key, along with appropriate zero-knowledge proof, is generated by the Election Provider (Polyas) and published on board keygen-electionKey-Polyas (Section E.6.2)

Cryptographic details of this process are given in Section A.7. The necessary verification step is to check correctness of the zero-knowledge proof published alongside the election key as described in Section A.7 (see Algorithm 12).

**Ballot box and ballot pre-processing.** Ballots are encrypted on the client side and published (via the voting server) on the ballot box. The voting server is supposed to publish only well-formed encrypted ballots and only one ballot for each voter. It means that invalid ballots should never be published on the ballot box. However, to transparently handle the case where (for any unanticipated reasons) an invalid or duplicated ballot gets published, we apply so-called ballot preprocessing process where invalid and/or duplicated ballots are, first, explicitly flagged and, then filtered out. Also at this point, if the election supports ballot revocations, the revoked ballots are flagged as well.

The cryptographic details of this process are given in Section A.5. The verification task here is to check that the invalid ballots are correctly flagged. To this end, the verification procedure needs to check correctness of the zero-knowledge proofs included in the ballot entry (see Algorithm 8).

**Creation of mix packages and verifiable shuffling.** The ballots are shuffled in packets of a fixed, predefined maximal size $K$ (also published on the registration board). Board mixing-input-packets (Section E.7.1) contains input mix packets, where all the eligible ballots, that is ballots not flagged as incorrect or revoked, are grouped in packets respecting the following rules:

1. the order of the ballots is preserved,
2. the maximal number of elements in a packet is $K$,
3. the minimal number of elements in a packet is $K/2$, unless the total number $n$ of correct ballots is smaller that $K/2$, in which case the size of the (only) packet

is $n$.

If different voters can vote for different ballot sheets, which is reflected in the fact that different voters have assigned different *public labels* (in short, a public label specifies the list of ballot sheets a voter is eligible to vote; see Appendix A.5 for more explanation), then the above process is done independently for each public label (each packet contains encrypted ballots of voters with the same public label, the order of ballots for each public label is preserved, and so on).

The verification task for this board is to check that the packets have been created correctly, that is preserving the listed rules.

The packets of ciphertexts published on board mixing-input-packets (Section E.7.1) are then cryptographically shuffled (packet-wise) and the shuffled (and re-encrypted) packets of ciphertexts are published on board mixing-mix-Polyas (Section E.7.2). The cryptographic details of the verifiable shuffling and the corresponding verification procedure are given in Section A.6.

**Verifiable decryption.** The process of verifiable decryption takes, as its input, the shuffled ciphertexts and decrypts them, providing additionally zero-knowledge proof that the decryption has been carried out correctly (without revealing the secret election key). The decrypted ballots are published (in packets) on board decryption-decrypt-Polyas (Section E.4.1).

Cryptographic details are given in Section A.8. Verification of this step requires an implementation of Algorithm 13 which checks the zero-knowledge proofs of correct decryption.

We want to note that Polyas Core 3.0 back-end routinely carries out the verification of all its subcomponents (including verification of the published zero-knowledge proofs) in order to make sure that the process proceeds as expected. Importantly for protecting ballot privacy, the decryption process (which decrypts the shuffled ballots) is only started after the content of all the preceding bulletin boards is fully verified. It means that the decryption key is used only after making sure that the input ciphertexts are exactly as expected. This is an important part of the policy of correct handling of the secret election key.

**Ballot decoding and final tallying.** Board decryption-decrypt-Polyas (Section E.4.1) contains decrypted ballots in binary format. These ballots are then decoded in order to carry out the final tally (which produces final, human-readable result). The verification of this step should re-calculate the final result based on the binary decrypted ballots

in order to check that the claimed result is correct. Details of this step are given in Section A.9.

**Additional verification tasks.** Additional verification tasks are concerned with integrity checks related to voters' receipts (signed ballot confirmations) and the set-up of the tools for cast-as-intended verifiability.

- When the election system is deployed and bootstrapped, it creates the so called *second-device public parameters*, that is parameters of the voting process relevant for the ballot receipt signing/verification and ballot audit (cast-as-intended verification with a second device). If a second device application is deployed, it should be pre-configured with this fingerprint.
  A tool for universal verifiability should offer the possibility of checking that a given file with the public parameters fingerprint is consistent with the content of the bulletin boards. This process is described in Appendix C.1.
- Voters have the option to save *ballot cast confirmations* (receipts), containing fingerprints of their encrypted ballots, signed by the election system. The voters can then hand over their receipts to auditors (who carry out the universal verifiability procedure), in order to make sure that their ballots are included in the final tally.
  To support this process, a verification tool needs to be able to process receipt files and make necessary consistency checks, as described in Appendix C.2.

**Additional information.** Appendix E contains technical details of all the involved bulletin boards. It includes also example data (taken from an actual, small system run). Appendix F lists and documents data types used in the system. Appendix D describes the format of the verification data packet.

# A Cryptographic Modules

In this section we provide details of the cryptographic building blocks used in the system.

## A.1 Auxiliary Algorithms

### A.1.1 KDF

In this section we describe a key-derivation algorithm, used to generate pseudo-random byte arrays from a given seed. Our implementation follows algorithm 5.1 from KDF in Counter Mode (NIST SP 800-108).

Based on this algorithm, we then define procedures that generate (from the given seed and in a reproducible way) numbers in the given range.

**The key derivation (KDF) procedure.** To generate the pseudo-random bytes from the given seed (and some additional, so called, domain parameters) we use Algorithm 1. This algorithm depends on a pseudo-random function. Following the NIST recommendation (which suggests using HMAC or CMAC), we use HMAC-SHA512 as our pseudo-random function (PRF), which we denote by *mac*.

**Algorithm 1:** Key derivation function

**Input :**

- the desired length $l$ (in bytes) of the pseudo-random output
- the initial seed $k$ (so-called *key derivation key*), given as a byte array,
- a byte array *label* that identifies the purpose of the derived key material
- a byte array *context* that, again, describes the context of the derivation procedure.

**Output :** Sequence of $l$ bytes computed in the following way:
Denote by $B_i$ the block of 64 bytes (512 bits) computed by:

$$B_i = mac_k(i \,||\, label \,||\, 0x00 \,||\, context \,||\, l)$$

where $i$ and $l$ above are represented as a 4-byte integer. (The NIST standard we refer to uses at this point the desired length *in bits*; here we define the procedure to operate on the byte-size granularity level.)
To produce the output, compute the necessary number of blocks $B_0, B_1, \ldots$ and take the first $l$ bytes of this sequence.

**Example:** For the initial seed set to "kdk" (more precisely, to the byte representation of this string using the UTF-8 encoding; we will use this convention throughout the following examples), the label set to "label", context set to "context", and the desired byte length $l$ set to 65, the above algorithm produces the following byte string (in the hexadecimal representation):

```
32 88 92 2A 96 65 33 C7 93 ED 53 20 45 FF FC 3C E6 BA 77 F2 7E 8F 60 C9 A3 D8 22 21 D8 6F 51 DD A0 07 36
DB A3 F8 AE 1D 94 B1 75 62 E8 38 D5 7F B8 54 00 D1 47 C6 E9 58 5E D4 D8 59 E4 61 20 B2 75
```

**Numbers from seed.** We describe now a method which produces a sequence of positive integers of a given bit length, generated deterministically based on the given seed and index, following algorithm A.2.3 Verifiable Canonical Generation of the Generator g from NIST fips186-4 (this method implements, in essence, a part of this algorithm). This method is used, in particular, to verifiable produce generators, as we will explain in the next section. The method is described in Algorithm 2.

**Algorithm 2:** Numbers from seed

**Input :**

- the desired length $l$ in bits of the generated numbers,
- the initial seed *seed*, given as a byte array,

**Output :** Sequence $n_1, n_2, \ldots$ of numbers in the range $[0, 2^l)$; the maximal length of the output sequence is limited by MAXINT32 which is the maximal number that can be represented as a 32-bit signed integer

1 **for** $i \in 1, \ldots$ **do**

2 Derive $\lceil l/8 \rceil$ bytes, using Algorithm 1, for the initial seed $k = seed \| i$, with $i$ represented as 4-byte integer, *label* being set to the byte representation of the string "generator", and *context* being set to the byte representation of the string "Polyas".

3 Ignore the appropriate number of left-most bits from the byte array above, to obtain a bit array $t_i$ of length $l$.

4 Define $n_i$ as the positive integer with the bit representation $t_i$. (*Implementation note: in some implementations, for example if the standard Java* `BigInteger` *class is used, one may need to add some leading zeros to the byte sequence in order to guarantee that the bit array will not be interpreted as a negative integer*).

**Example:** The first three numbers generated using Algorithm 2 for the initial seed "xyz" and $l = 520$ are

$a_1$ = 1732501504205220402900929820446308723705652945081825598593993913145942097001127020633138020218038968109094917857329663184563374015879596834703721749398989648

$a_2$ = 22074013036655034340315313555119229748896928176011835002592637426259140610461461429293767780728274504619363005332069049797404744820588400037203799604910235 11

$a_3$ = 18838895879035194773578385142239539799542013446656817983670231963287219757200521539135821221519137852732229217868898369877312967288251196048096094101579874 02

$a_4$ = 14232598494672177111858747995156078428426027857678797666237362846802098327046383909004125971969487500159762717939307137448905476116550648351658833233889981463

The above algorithm can be easily adopted to generate a sequence of numbers from a given range $[0, b)$, as specified in Algorithm 3.

**Algorithm 3:** Numbers from seed (range)

**Input :** Positive integer $b$ and an initial seed *seed* given as a byte array
**Output :** Sequence $n_1, n_2, \ldots$ of numbers in the range $[0, b)$

1 Use Algorithm 2, with the initial seed *seed*, to generate a sequence of positive integers of the bit length $l$, with $l = \lceil \log_2(b) \rceil$.
2 Discard all the elements which are not in the desired range $[0, b)$.

**Example:** The first two numbers generated using Algorithm 3, for the initial seed "xyz" and $b$ set to $a_1 + 1$ (where $a_1$, $a_2$, $a_3$, $a_4$ are as in the previous example) are $a_1$ and $a_4$ (note that $a_1$ and $a_2$ are not included).

### A.1.2 Hashing into $\mathbb{Z}_q$

In several cryptographic constructions involving ElGamal groups, we need a hash function which, for given input data, returns an element of $\mathbb{Z}_q$ (for some number $q$). To this end, we use the SHA-512 algorithm as the building block and follow the following conventions.

The **input data** (if not directly given as a byte array) before being fed into SHA-512, is transformed into a byte array in the canonical way. In particular:

- Strings are transformed to a byte array using the "UTF-8" encoding.
- Integers, if not explicitly stated otherwise, are represented as 4 bytes in the big-endian byte order (with the most significant byte coming first).
- Big integers represented by Java class `BigInteger` are transformed to byte array using the following method. One takes the byte representation of $b$ of the given big integer, as obtained by calling method `toByteArray()`, and prepends it with 4 bytes representing the length of $b$.
  For instance, the number 98162874527223464716009286152 gets transformed to byte array `00 00 00 0d 01 3D 2E 6D 3A FD EC 0F 0A 00 AD 2A 08`.

- Elliptic curve points are transformed into a byte array using the compressed form of ANSI X9.62 4.3.6.[1]
  For instance, the point

  (`75788b8a22a04baad44c66ec80e86928597979bf1b287760ad4e3153293d613b`,
  `664663757d16eff0b993ac12a1ba16ee4784ac08206b12be50f4d954d9d74c88`)

  (of the `secp256k1` curve) gets transformed to byte array
  `02 75 78 8B 8A 22 A0 4B AA D4 4C 66 EC 80 E8 69 28 59 79 79 BF 1B 28 77 60 AD 4E 31 53 29 3D 61 3B`
- For an ElGamal ciphertext $(x, y)$ (as for pairs in general), we first digest the first component $x$ and then the second component $y$.

Note that the order of the input elements is significant: the elements must be digested always in the specified order.

Considering hashing into $\mathbb{Z}_q$, we distinguish two cases:

- **Non-uniform hash.** If it is not relevant that the distribution be uniform, we apply the SHA-512 function to the input data (using the described above conventions), obtaining the byte array $h$ and then transform it into a number by constructing a Java big integer object directly from $h$ and modulo $q$: `(new BigInteger(`$h$`)).mod(`$q$`)`.
- **Uniform hash.** This method, presented in Algorithm 4, distributes the result of the hash function (pseudo)-uniformly in $\mathbb{Z}_q$.

**Algorithm 4:** Uniform Hash

**Input** : Positive integer $q$ and input data to be digested
**Output**: A number in the range $[0, q)$

1 Apply the `SHA-512` function to the input data (using the above conventions), obtaining a byte array $h$.
2 Use $h$ as the seed to Algorithm 3 with parameter $b = q$ and
3 **return** the first number of the sequence generated by this algorithm.

We will denote application of Algorithm 4 with parameter $q$ to some data $a_1, \ldots, a_n$, where $a_i$ are elements that can be converted to byte arrays, using the conventions described above, by $\mathit{uniformHash}_q(a_1, \ldots, a_n)$.

**Example:** The uniform hash algorithm, for $q = 2126991829$ and input data `"some data"` returns 414907466.

[1] Or equivalently https://www.secg.org/sec1-v2.pdf in section 2.3.3. In the case the BouncyCastle library is used, this is achieved for an elliptic curve point `p`, by calling `p.getEncoded(true)`.

**Example:** For $q = 2126991829$, $s =$ `"some data"`, and $n = 98162874527223464716009286152$ (where $n$ is of type `BigInteger`), the result of $\mathit{uniformHash}_q(s, n)$ is 1444258901.

### A.1.3 Encoding Plaintexts as Numbers

Before a given plaintext messages (byte array) *msg* can be encrypted using the ElGamal encryption scheme, it needs to be represented as a sequence $a_1, \ldots, a_n$ of integers in the range $[0, q)$ (note that in the general case, it may be not possible to represent the plaintext message as a single integer in this range), as detailed below. Such a sequence is called a *multi-plaintext* (as it consists of possibly several simple plaintexts $a_1, \ldots, a_n$).

Such an encoding is presented in Algorithm 5. We note that this algorithm is presented here for completeness and does not have to be implemented for the verification task which only requires the corresponding decoding algorithm presented next.

The reverse operation—that is decoding a given multi-plaintext back as a message—is presented in Algorithm 6.

**Example.** For $q = 2^{32} - 1$ and the message *msg* = `"qwertyuioplkjhgfdsazxcvbnm"` (that is the byte representation of this string), Algorithm 5 returns multi-plaintext

625, 7824754, 7633269, 6909808, 7105386, 6842214, 6583137, 8026211, 7758446, 7143424

This multi-plaintext is mapped by Algorithm 6 back to *msg*.

## A.2 Used ElGamal Group

In the variant of the Polyas system described in this documentation, we use the ElGamal group based on the elliptic curve `secp256k1` [1]. This curve is defined over the finite field $F_p$ with

$$p = \texttt{fffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f}$$

($p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$) and has coefficients $a = 0$ and $b = 7$. The generator of the group is

$$g = (\texttt{79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798},\\ \texttt{483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8})$$

This group is of prime order

$$q = \texttt{fffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141}$$

**Algorithm 5:** Encoding a message as a multi-plaintext

**Input :** A positive number $q$ and a byte array $msg$ with consecutive bytes $b_1, \ldots, b_l$

**Output :** Encoding of $msg$ as a multi-plaintext $a_1, \ldots, a_m$

1 Compute the block size $s = \left\lfloor \frac{\log_2(q)}{8} \right\rfloor$. (This value expresses how many bytes can be represented as one number in the range $[0, q)$; more precisely, $s$ bytes can be represented as a number in the range $[0, u)$, where $u = 2^{8s}$; note that $u \leq q$.)

2 Compute a padded version $msg'$ of $msg$ by prepending it by two bytes containing the length of the pad and appending the pad, where the pad consists of zero-bytes and the length $k$ of the pad is the minimal integer such that the size of $msg'$ is dividable by the block size $s$. That is, $msg'$ consists of bytes

$$c, c', b_1, \ldots, b_l, z_1, \ldots, z_k$$

where each $z_i$ (for $i \in \{1, \ldots, k\}$) is a zero byte, the bytes $c, c'$ contain the value $k$ (the pad length encoded in two bytes using the *big endian* byte order (where the most significant byte is in the left-most one), and the length of the pad $k = \left\lceil \frac{l+2}{s} \right\rceil \cdot s - (l + 2)$. Note that the length of $msg'$ is $\left\lceil \frac{l+2}{s} \right\rceil \cdot s$ which is indeed dividable by $s$.

3 Repeat the following step: take $s$ consecutive bytes of $msg'$ and convert them to a positive integer (such an integer will be in the range $[0, q)$ by the definition of $s$), using the *big-endian* byte-order. Note that, for some implementations one needs to be make sure that the leading bit is zero, in order to obtain positive integers. By repeating this step we obtain consecutive elements $a_i$.

**Algorithm 6:** Decoding a message from a multi-plaintext

**Input :** A positive number $q$ and a multi-plaintext $a_1, \ldots, a_m$, where $a_i \in [0, q)$

**Output :** A byte array $msg$

1 For each number $a_i$ obtain its representation as a byte array of length $s$, where $s$ is like in Step 1 of Algorithm 5. Let $msg'$ be the concatenation of the byte arrays obtained in the above way for all the numbers from $a_1$ to $a_m$.

2 Interpret the first two bytes of $msg'$ a positive integer $k$.

3 Check that $k$ right-most bytes of $msg'$ are zeros.

4 Truncate two left-most bytes and $k$ right-most bytes of $msg'$ to obtain $msg$.

5 **return** $msg$.

### A.2.1 Elliptic curve encoding

In order to encrypt plaintexts, which are represented as numbers in $\mathbb{Z}_q$, using the ElGamal encryption scheme over the given cyclic group, one first needs to convert the given plaintext into an element of the group; conversely, decryption requires converting a group element back into $\mathbb{Z}_q$. However, unlike the other common groups used in cryptography, in general there exists no efficiently computable homomorphism between the integers modulo $q$ and an elliptic curve of order $q$.

We use therefore (in a slightly modified way) a method suggested by Neal Koblitz in Elliptic Curve Cryptosystems [6]. We note that a very similar method is used in the *Verificatum Core Routines*.

The method will use a constant $k$ which, by default, we set to 80. Let $\mathit{messageUpperBound} = \lfloor p/k \rfloor$ (recall that $p$ is the order of the underlying finite field).

The encoding works for positive integers $a < \mathit{messageUpperBound}$ and succeeds with overwhelming probability in $k$. Notice that this encoding is somehow suboptimal, because the message space is limited to $u$ rather than the order of the group $q$, but it is not of practical significance.

**Encoding:** Given a number $a$ to be encoded, iterating a counter $i$ from 1 to $k$, we set the $x$-coordinate to be $x_i = a * k + i \bmod p$. We then solve the elliptic curve equation in the standard way (using the Tonelli-Shanks algorithm to compute square roots modulo a prime) to compute $y_i$ (if such a solution exists) such that the point $(x_i, y_i)$ is on the curve. If so, we return this point; otherwise we continue to iterate. If after iterating $k$ times no solution is found, the algorithm fails.

Under the (commonly accepted) assumption that the $x$-coordinates of the group elements are uniformly distributed in the underlying finite field, the probability of failing to encode a given number $a$ is approximately $\left(1 - \frac{q}{2p}\right)^k$.

**Decoding:** Given an elliptic curve point $(x, y)$, we compute the message $a = \lfloor (x - 1)/k \rfloor$.

**Example:** For

$a = 72370055773322622139731865630429942408293740416025352524660990004943021669 8$

the above encoding yields the point

(7fffffffffffffffffffffffffffffffffffffffffffffffffffffff7ffffe21,
2af4d53f09f4d4ede3caf3f0e06ccfc0f55289d83fed859ca504d6033bec629b)

that is the point with the $x$-coordinate $x = a * 80 + 1$.

### A.2.2 Select Generator Verifiably

We describe now a method for generating group elements from a given seed, inspired by Algorithm A.2.3 *Verifiable Canonical Generation of the Generator* from NIST fips186-4. When used with a (pseudo-)random seed, this method produces pseudo-random group element distributed (pseudo) uniformly in the group. When used with a pre-agreed seed, this method can be used to select independent generators of the given group in a verifiable (reproducible) way. The latter is crucial, in particular, for integrity of verifiable shuffling.

The mentioned standard covers the case of the traditional Schnorr groups (of quadratic residues modulo a safe prime); the method presented here is an adaptation of this algorithm for the elliptic-curve-based groups of prime order.

The method is presented in Algorithm 7.

**Algorithm 7:** Independent generators for EC groups of prime order

**Input :**

- the group parameters: the prime order of the underlying field $p$, coefficients $a$ and $b$ of the elliptic curve,
- the seed *seed*,
- an integer *index* which specifies the index of the generator to be obtained from that seed.

**Output :** The *index*-th generator for the given parameters.

1 Let $w_1, w_2, \ldots$ be the sequence of numbers in the range $[0, 2p)$ generated from the seed $s = (seed||\text{"ggen"}||index)$, where *index* is represented as 4-byte integer, using Algorithm 3.

2 **for** $w$ *in* $w_1, w_2, \ldots$ **do**

3 Define $x := w \mod p$.

4 Try to compute (using for instance the Tonelli–Shanks algorithm), the square root $\hat{y} = \sqrt{x^3 + ax + b} \mod p$.

5 **if** *the square root* $\hat{y}$ *exists* **then**

6 Take $y = (-\hat{y} \mod p)$, if $w < p$, and $y = \hat{y}$ otherwise (so we use the most significant bit of $w$ to determine which of the two possible $y$-coordinates should be taken).

7 If the obtained point $(x, y)$ is a generator of the group (which holds always except for the point at infinity), then **return** this point (otherwise, continue to the next number $w$).

**Example:** The numbers returned by this algorithm for the `secp256k1` curve with *seed*

set to the (byte representation of the string) "seed", and for *index* in the range $1, \ldots, 3$ are:

```
(879f580dfe31c74dc2b4289f1988e581c76e625761a863971c808e90ab6fd3c7,
   4c59d9061d35678d06c04fe9f61dd47d7ee9e35b9847e5f3f9ed532c509afc0f),
(b6413eb866319a631509ad0e637ec260507383d7495ef66858f9a6a4bb8efac7,
   adb88f8cdd62aad64e2518d383b4e6aa2013910964b6423c17c0100f96118ae1)
(1845cc619ec1a70c743e6559938290b7dac3d63b3fd2cf8d6e0646d292a576e8,
   439f7d97af4396e0441b7d292045cf78bc22187eec981e5d6fe2ebd0794f975a)
```

## A.3 Everlasting Privacy

The property of everlasting privacy means that — under reasonable assumptions — the secrecy of each ballot is protected indefinitely. This guarantee holds even if the cryptographic techniques used today become vulnerable or compromised at some point in the future. In technical terms, everlasting privacy means that ballot secrecy is preserved (again, under reasonable assumptions) even against a computationally unbounded adversary, meaning an attacker with unlimited computing resources.

Polyas Core 3 e-voting guarantees everlasting privacy, while at the same time maintaining other relevant integrity properties, such as end-to-end verifiability. It is achieved using the following method.

To protect voter anonymity, encrypted ballots stored in the digital ballot box do not include voter identifiers. The system removes any link between a stored ballot and the voter, ensuring that no one — *except the voter themselves* — can associate one to the other. More specifically, the association between the given ballot and the voter identity can only be established using a dedicated secret, that we will call a *linking token*, which is assumed to be only known to the voter. Without this token, it is impossible for anyone — even an attacker with unlimited computational resources — to link a ballot to its voter.

In the following, we describe this mechanism in more detail.

**Credential and registry generation (the general case).** In the registration phase, the following data is generated (by the Registrar), for each voter $i$ with identifier $v_i$:

- A *secret credential* $s_i$ is randomly sampled from $\mathbb{Z}_q$. The corresponding public credential is computed as $p_i = g^{s_i} \mod p$. (See Section A.2 to recall the setup.) The secret credential will serve as the secret key, used by the voter to sign their ballot. The corresponding public credential will be used to check the signature.
- An *anonymous voter's reference* $ref_i$ is computed as a perfectly hiding commit-

ment to the voter's identity, using a random value $\ell_i \in Z_q$, called the *linking token*. That is $ref_i = \mathsf{Comm}(v_i, \ell_i) = g^{v_i} \cdot h^{\ell_i}$.

The secret credential $s_i$ and the linking token $\ell_i$ are securely delivered to the voter. The public values $(p_i, ref_i)$ are added to the *public registry*. The registry entries are published in a randomized order (or sorted) and do not include the voter identifiers. The voters may also receive separate login credentials from the voting server.

**Ballot preparation and casting.** In the voting step, the voter provides their login credentials, along with the values $s_i$ and $\ell_i$ to their voting client application. The client application creates a ballot containing the encrypted voter's choice and the anonymous voter's reference $ref_i$. The application signs this ballot with the secret voter's key $s_i$. See Section A.5 for more details of the ballot preparation process.

The ballot created in the above way is submitted to the voting server together with the linking token $\ell_i$. This allows the voting server to checks consistency of the voter's reference $ref_i$ included in the ballot with the voter's identity established via the authentication process. Importantly, the linking token $\ell_i$ is only used during the ballot cast time and is not published nor persisted in any way by the voting server. Therefore, once the ballot is stored in the ballot box, the link to the voter identity can only be recreated/demonstrated by the voter who is assumed to remain the only party in the possession of their linking token.

**Ballot audit with second device.** The mechanism for everlasting privacy described here is compatible with the cast-as-intended method with second device, presented in Section B. In particular, the linking token enables the voter to make sure that the audited ballot is uniquely associated to him/herself. Specifically, the voter client includes the linking token $\ell_i$ in the QR-code that is given to the second device (ballot audit application). The second device application requests then the ballot with the corresponding reference $ref_i$ from the election system for audit. Using $\ell_i$, the second device application can make sure that the audited ballot is uniquely bound to the identity of the voter (this check effectively prevents the so-called clash attacks [7]). See details of this process in Section B.

**Trust assumptions.** The described here method provides everlasting privacy, based on the assumption that the linking tokens are only known to the voters themselves. This translates into the following more detailed trust assumptions:

- *The Registrar honestly carries out the registration phase*: it does not store nor leak the linking tokens it generates for the voters.
- *The voting server honestly carries out the ballot cast procedure*: it uses the linking token provided by the voter, according to its purpose, to check the integrity of

the ballot and does not store and leak it in any way. The voting server also does not log the association between the voter and the cast ballot.

- *The second device — if used by the voter — acts according to the specification*: it should not store nor leak the linking token which is included in the QR-code.

When the above trust assumptions are met, the everlasting privacy is guaranteed by the perfectly hiding property of the used commitment scheme.

Note that these assumptions are only required for the everlasting privacy, but not for the "standard" ballot privacy. The difference is that everlasting privacy (making the stronger assumption about the capabilities of the adversary) postulates that even the link between the voter's identity and their *encrypted* ballot cannot be seen. In this sense, everlasting privacy adds stronger guarantees on top of the standard ballot privacy which relies on the encryption and the cryptographic anonymisation method (verifiable shuffling).

It is also worth noting that this kind of trust assumptions is common for everlasting privacy methods known from the literature. That is, it is common that for everlasting privacy one puts trust is some parts (roles) of the election system.

## A.4 Credential Generation

In this section, we describe the process of generating voters' credentials (passwords). We consider here a *simplified setup*, where the secret credentials $(s_i, \ell_i)$ and the authentication passwords are generated by the same party (the Registrar).

Note that the credential generation methods described in Section A.3 works also for more complex setup, where the authentication credentials are generated separately. In fact, for elections with the highest security requirements, such a more complex setup may be recommended. However, even with the simplified setup considered in this section, the voting server may use additional authentication factors (such as one-time codes sent to voters during the authentication process) achieving very high security level of the authentication process.

The process of credential generation takes, as its input, the following data:

- a list $(id_1, address_1), \ldots, (id_n, address_n)$ of voter identifiers and their addresses (these addresses will be used by the distribution facility to deliver credentials to voters),
- the public encryption key of the distribution facility.

The credential generation process produces (i) data for the distribution/printing facility containing the confidential information which needs to be securely delivered to the voters and (ii) data for the election system, containing the public registry. It proceeds as follows.

- Following Section A.3, for each voter $i$, a *secret credential* $s_i$ is randomly sampled from $\mathbb{Z}_q$ and the corresponding public credential is computed as $p_i = g^{s_i} \mod p$.
- Again following Section A.3, for each voter $i$, the *linking token* $\ell_i$ is randomly sampled from $\mathbb{Z}_q$ and the corresponding anonymous reference is computed as $ref_i = g^{v_i} \cdot h^{\ell_i} \mod p$.
- Additionally, for each voter $i$, a passcode $\tau_i$ is randomly generated.
- From the passcode $\tau_i$ the following values are derived (using a key-derivation function(s)):
  - the *login password*,
  - a symmetric encryption key $k_i$.

  The hash $h_i$ of the login password is then computed.
- The values $s_i$ and $\ell_i$ are encrypted with the key $k_i$. We will denote the result of this encryption as $\bar{s}_i$ and $\bar{\ell}_i$, respectively.

**Data delivered to the election system** consists of two parts. The first part contains the list of records of the form

$$id_i, h_i, \bar{s}_i, \bar{\ell}_i.$$

Values $id_i, h_i$ are intended to be used for voter authentication, while values $\bar{s}_i, \bar{\ell}_i$ play a part in the secret delivery mechanism, as described below. The second part contains reordered (by sorting) list of records of the form

$$p_i, ref_i.$$

These values constitute the content of the public registry which is published on one of the secure bulletin boards (see section E.1.2).

**The data delivered to the distribution facility** consists of the records of the form

$$address_i, \tau_i$$

The delivery facility is supposed to securely deliver the passcode $\tau_i$ to the corresponding address.

**Ballot cast.** During the ballot cast process, the voter enters their passcode $\tau_i$ to the voter client. From this passcode, both the login password and the symmetric key $k_i$

are derived. The login password is used for authentication (note that the voting server knows the corresponding hashed password). After successful authentication (which may also involve additional authentication factors), the voting server responds with the encrypted values $\bar{s}_i$, $\bar{\ell}_i$, from which the voter client decrypts $s_i$ and $\ell_i$, using the key $k_i$ and proceeds to the ballot creation and casting, as shortly described in Section A.3 and in more detail in Section A.5.

Note that with this solution, the passcode $\tau_i$ is the only secret delivered to the voter. This addresses a potential usability issue that would arise if the values $s_i$ and $\ell_i$ were directly sent to the voter, as these values are relatively big and may be, in particular, cumbersome to type in.

***Note.*** *The public registry contains some additional data including, in particular, the so-called* public labels *of the voters. See Section A.5.1 for the explanation of this notion. The detailed format of the data published on the public registry board can be found in Section E.1.2. .*

## A.5 Ballot Creation and Validation

In this section, we describe the intended way in which ballots are created and how they should be verified.

Ballots are encrypted on the client side, submitted via the voting server, and published in the ballot box. The voting server is supposed to publish only well-formed encrypted ballots (that is ballots with valid zero-knowledge proofs, as specified below) and only one ballot for each voter. It means that invalid ballots should never be published on the ballot box. However, to transparently handle the case where (for any unanticipated reason) an invalid or duplicated ballot gets published, the Polyas Core 3.0 back-end first applies so-called *ballot preprocessing* process where invalid ballots are, first, explicitly flagged; only the ballots which are not flagged as incorrect are then taken for the further tallying. We emphasise that, for transparency, ballots are never silently removed from the ballot box, but, instead, incorrect ballots are explicitly handled as explained.

### A.5.1 Public labels

Ballots published on the ballot box include a special field called a *public label*, see BallotEpEntry (Section F.4), field `publicLabel`. A public label contains the public information about the ballot, namely which ballots a voter is eligible to vote for. For instance, a public label `"A1:A3:B2"` means that the given voter is eligible to vote for three ballot sheets with identifiers `A1`, `A3`, and `B2` and, therefore, a ballot with such a label is supposed to contain encrypted votes for these three ballot sheets. A public label for a voter (and hence which ballot sheets the voter is eligible to vote for) is

specified on the registry board (see Section E.1.2).

A public label is not only attached to the ballot in the ballot box, but it is propagated throughout the whole system. For instance, mix packets, that is packets of ballots to be shuffled in a verifiable way, group ballots with the same public label (the same ballot sheets). Finally, the public label is propagated to the final board containing decrypted ballots, where it provides an important context: to properly decode and tally decrypted ballots, one needs to know which ballot sheets are encoded in a given ballot; the public label provides this information.

If, in a given election, all voters vote for the same ballot sheets, then all the ballots will be labeled with the same public label.

### A.5.2 Ballot encoding, encryption, and casting

In order to create an encrypted ballot, the following input is used:

- the election public key $pk \in G$ (published on the board keygen-electionKey-Polyas (Section E.6.2)),
- the plaintext voter's choice *msg*, given as a byte array (such a plaintext choice is created by encoding the voter's choice using algorithm presented in Appendix A.9),
- the public label *lab* of the voter, given as string (recall that a public label is a public information assigned to the voter which specifies which ballot sheets a voter votes for),
- the voters (public) unique identifier *vid*,
- the linking token $\ell \in Z_q$ (which is an opening to the anonymous voter's reference *ref*).
- the private credential $s$ (private signing key) of the voter.

This procedure also uses (implicitly) the fixed ElGamal group $G$ of order $q$ with a fixed generator $g$ and a fixed independent generator $h$ used for Pedersen commitments, as introduced above.

In the first step, the plaintext message *msg* is represented as a sequence $a = a_1, \ldots, a_n$ of integers in the range $[0, \mathit{messageUpperBound})$, a so-called *multi-plaintext* (note that in the general case, it may be not possible to represent the plaintext message as a single integer in this range), using Algorithm 5 with parameter $q$ set to *messageUpperBound*.

Having computed the multi-plaintext $a = a_1, \ldots, a_n$ as described above, we encrypt each simple plaintext $a_i$ of this multi-plaintext, using the standard ElGamal encryption,

obtaining a so-called *multi-ciphertext* $e = e_1, \ldots e_n$:

$$e_i = (g^{r_i},\ \gamma(a_i) \cdot pk^{r_i})$$

where $r_i$ are freshly generated random encryption coins from the set $\mathbb{Z}_q$ and $\gamma(a_i)$ denotes the encoding of $a_i$ as an element of the group $G$, as specified in Section A.2.1. We will refer to the multi-ciphertext $e$ as the *encrypted choice*.

Next, we compute the *anonymous voter's reference* $ref \in G$ which is a Pedersen commitment to the voter's identifier *vid*, using the linking token $\ell$ as the random coin:

$$ref = g^{v} \cdot h^{\ell} \tag{1}$$

where $v = \mathit{UniformHash}_q(vid)$. The reference *ref* serves as a unique, anonymous identifier of the voter, where the underlying voter's identity *vid* remains hidden under the perfectly hiding commitment with the randomness $\ell$.

Now, we compute *the proof of the knowledge of the encryption coins*, which is the non-interactive version of the well-known zero-knowledge proof of knowledge of the discrete logarithm ($r_i$) of the first component of each $e_i$ [10]. This proof is of the form $\pi_1 = (c_1, f_1), \ldots, \pi_n = (c_n, f_n)$ and is generated, for each $i \in \{1, \ldots, n\}$, by drawing a random element $b_i$ from the set $\mathbb{Z}_q$ and computing

$$\begin{aligned} B_i &= g^{b_i} \\ c_i &= \mathit{UniformHash}_q(g, pk, lab, ref, e_1, \ldots, e_n, B_i) \\ f_i &= b_i + c_i r_i \mod q \end{aligned}$$

Finally, the ballot creation procedure creates a Schnorr signature $\sigma = (c, f)$ on the ballot, using the secret voter's key $s$, by drawing a random element $b$ from $\mathbb{Z}_q$ and computing

$$\begin{aligned} B &= g^{b}, \\ c &= \mathit{UniformHash}_q(g, pk, lab, ref, e_1, \ldots, e_n, B), \\ f &= b + cs \mod q. \end{aligned}$$

The ballot, as submitted by the voter's client and as published in the ballot box, consists of:

$$b = (e_1, \ldots e_n,\ \pi_1, \ldots \pi_n,\ lab,\ ref,\ \sigma).$$

The voter's client application (voter's agent) passes the ballot $b$, together with the linking token $\ell$ to the election system.

### A.5.3 Server-side ballot validation

The election system carries out the following validation steps:

- it checks the consistency of the identity *vid* of the authenticated voter (we assume here that the election system has already carried out voter's authentication) with the reference *ref* included in the ballot, using the provided linking token $\ell$, by checking that Equation (1) holds for these values,
- it checks that no ballot with the same *ref* have been already published in the ballot box (in this variant of the system, we do not support revoting/ballot update).
- it validates the ballot, as specified in Algorithm 8, using the voter's public credential corresponding to the public reference *ref*, as specified in the public registry.

The ballot is published in the ballot box only if the above steps succeed. Furthermore, the linking token $\ell$ is used by the election system only for this validation step and is not persisted in any way (it is not available to the election system once the ballot has been published).

### A.5.4 The verification task

The (universal) verification task is supposed to check that the flagging process, as introduced above, is done correctly (and report any incorrect entries). A ballot should be **flagged as incorrect** in the following cases:

- There has been a ballot with the same reference *ref* published before.
- The ballot is not well-formed, i.e. Algorithm 8 fails for it.

Note that, as opposed to the validation steps carried out by the election system on the ballot cast, the universal verifiability steps do not include checking the consistency of the voter's identity and reference *ref* included in the ballot. In fact, neither the voter identity nor the linking token $\ell_i$ are not know at this time.

**Computing the expected length of an encrypted ballot.** In Step 2 of Algorithm 8, we need to compute the expected length of an encrypted choice for the given public label. To determine this length, one should (1) take the expected length of the encoded plaintext and (2) compute the length of resulting ciphertext.

**Algorithm 8:** Verification of a ballot

**Input:**

- public parameters: parameters of the used group $G$, including the independent generators $g, h$, and the public election key $pk \in G$,
- ballot $b = (e_1, \ldots e_n, \pi_1, \ldots \pi_n, \mathit{lab}, \mathit{ref}, \sigma)$, where $e_i = (x_i, y_i) \in G^2$, $\pi_i = (c_i, f_i) \in Z_q^2$, $\mathit{ref} \in G$, and $\sigma = (c, f) \in Z_q^2$,
- voter's public credential $\alpha \in G$, as associated to the given reference *ref* through the public voters' registry.

1 Check that the public label *lab* (included in the ballot) is the same as the public label specified in the voters' registry under the reference *ref*.

2 Check that the length $n$ of the encrypted choice is as expected for this public label (see below).

3 **for** $i \in \{1, \ldots, n\}$ **do**

4 Check that $c_i = \mathit{UniformHash}_q\left(g, pk, \mathit{lab}, \mathit{ref}, e_1, \ldots, e_n, \frac{g^{f_i}}{x_i^{c_i}}\right)$.

5 Check that $\sigma = (c, f)$ is in fact a valid signature on the ballot w.r.t. the public credential $\alpha$, by verifying that

$$c = \mathit{UniformHash}_q\left(g, pk, \mathit{lab}, \mathit{ref}, e_1, \ldots, e_n, \frac{g^f}{\alpha^c}\right)$$

In order to (1) compute the expected length of the encoded plaintext, the following information is used: (a) the public label of the voter, as included in the ballot, which specifies identifiers of ballot sheets $(b_1, \ldots, b_l)$ the voter votes for and (b) the specification of the ballots, also included in the registry board. Given this information, one can compute the expected length of an encoded plaintext which contains choices for the ballots $b_1, \ldots, b_l$. The algorithm of ballot encoding is presented in Appendix A.9. As one can see, the expected length of the encoded plaintext is derived from the ballot structure (the number of voting options) in a straightforward way.

Now, in order to (2) compute the length of the resulting ciphertexts, one can construct any plaintext *msg* of the length $k$ determined in the step above (for instance a plaintext *msg* contained $k$ zero bytes) and take the length of the *multi-plaintext* computed from *msg*.

## A.6 Verifiable Shuffle

In this section we describe the way in which ballots are shuffled and how this shuffle can be verified. We use construction proposed by Wikströmet al. [11, 12] which is used, amongst others, in Verificatum [13]. Specifically we take the optimised variant for ElGamal as it is presented in Haenni et al. pseudocode algorithms for implementing Wikström's verifiable mix net [4], while extend the construction to support parallel shuffles (where one can shuffle *multi-ciphertexts*, that is ciphertext consisting of a number of simple ElGamal ciphertexts). For completeness, we provide the proofs in our technical report [5].

***Note.*** *The packets to be shuffled are read from board mixing-input-packets (Section E.7.1). The shuffled packets are published on board mixing-mix-Polyas (Section E.7.2). The verification procedure should check that the output mix packets contain valid zero-knowledge proofs, as described below, with respect to the corresponding input packets. .*

A verifiable shuffle in the simple case (that is without parallel shuffles), takes a list of ciphertexts, which it re-encrypts and shuffles to produce an output list of ciphertexts. More specifically, a cryptographic shuffle of ElGamal encryptions $\mathbf{e} = (e_1, ...., e_N)$ is another list of ElGamal encryptions $\mathbf{e}' = (e'_1, ..., e'_N)$, which contains the same plaintexts in permuted order. Additionally, a party that carries out shuffling produces an evidence to prove that input and output ciphertext are in fact in the described relation. This called a *proof of shuffle*. In the extended case (that we consider in this section) $e_i$, $e'_i$ are multi-ciphertexts (sequences of ElGamal ciphertexts of some fixed length).

The algorithm assumes the following **setup**:

- a cyclic group $G$ of prime order $q$ in which both the decisional and computational Diffie-Hellman problems are hard,
- the public key $pk$,
- a multi-commitment key $ck$, containing independent generators $h, h_1, ..., h_N \in G$.

**Selecting independent generators.** It is critical, that the elements of the multi-commitment key are indeed generated independently (security of extended Pedersen commitments depends on the assumption that one does not know/cannot compute the discrete logarithm of one element of the commitment key in the basis of another one). To this end, we generate the multi-commitment key in the reproducible and pseudo-random way, as follows:

$$\begin{aligned} h &= gen(10) \\ h_i &= gen(10+i) \qquad \text{for } i = 1, \ldots N, \end{aligned}$$

where $gen(j)$ is the generator obtained using Algorithm 7 with parameter *index* set to $j$ and parameter '*seed*' set to (the byte representation of the string) `"Polyas"`. Note that Algorithm 7 works for elliptic curve based groups of prime order, such as the group `secp256k1` we use in our system instance; for different groups, analogous methods should be used.

**Example.** For the used `secp256k1` group, the first few elements of the multi-commitment key, generated as above, are (represented in the compressed form of ANSI X9.62 4.3.6):

$h$ = `02549196EA21197151C73C3C9BDA1F12DA2BBEA99F2EFB0DD8BC235A9CED37ECB9`

$h_1$ = `03F08FCB284F32B737E0529840334D481E055AD6AFA18AB91A3B02939EB19EB8DD`

$h_2$ = `034A90A88BD2D3A92D7A29D19135F25536516D46FE4B8776C74B9E26D834FDA588`

$h_3$ = `0365DB947FD33BE257599D9E0BD1513E6F7B3BBE6C9008382E22F4B527D3A39299`

$h_4$ = `031E9073F6821FADE4307507F0D2756EFAAA4522FC15391390C943F4F4D9F32CE5`

The **interactive zero knowledge proof** of the verifiable shuffle, run by the prover $\mathcal{P}$ and the verifier $\mathcal{V}$, is given in Algorithm 9, where we use the following notation.

- We use the multiplicative notation for the group operation. As usually, by $\mathbb{Z}_q$ we denote the field of integers modulo the prime $q$.
- $A^N$ denotes the set of vectors of length $N$ containing elements of $A$. We will typically denote vectors in bold, for instance $\mathbf{a}$. We will denote the $i$-th element of such a vector using subscript, for instance as $\mathbf{a}_i$.
- Similarly, $A^{N \times N}$ is the set of square matrices of order $N$ containing elements of $A$. We will denote matrices using upper case letters, for instance $M$. By $M_{i,j}$ we denote the element of $M$ in the $i$-th column and $j$-th row.

**Algorithm 9:** Interactive ZK-Proof of Extended Shuffle

**Common Input** : group generator $g \in G$, public key $pk \in G$,
multi-commitment key $h, h_1, ..., h_N \in G$,
matrix commitment $\mathbf{c} \in G^N$,
ciphertext vectors $\mathbf{e}_1, ..., \mathbf{e}_N \in (G^2)^w$ and
$\mathbf{e}'_1, ..., \mathbf{e}'_N \in (G^2)^w$.

**Private Input of $\mathcal{P}$** : Permutation $\pi$ of the set $\{1, \ldots, N\}$, randomness $\mathbf{r} \in \mathbb{Z}_q^N$ and randomness $R \in \mathbb{Z}_q^{N \times w}$, such that $\mathbf{c} = Com(M_\pi, \mathbf{r})$ and $\mathbf{e}'_i = \mathbf{ReEnc}_{pk}(\mathbf{e}_{\pi(i)}, R_{\pi(i)})$.

1 $\mathcal{V}$ chooses $\mathbf{u} \in \mathbb{Z}_q^N$ randomly and sends it to $\mathcal{P}$.

2 $\mathcal{P}$ computes $\mathbf{u}' = M\mathbf{u}$. Then $\mathcal{P}$ chooses $\hat{\mathbf{r}} \in \mathbb{Z}_q^N$ at random and computes

$$\bar{r} = \mathbf{r}_1 + \cdots + \mathbf{r}_N, \qquad \tilde{r} = \langle \mathbf{r}, \mathbf{u} \rangle,$$
$$r^\diamond = \hat{\mathbf{r}}_N + \sum_{i=1}^{N-1} \left( \hat{\mathbf{r}}_i \prod_{j=i+1}^{N} \mathbf{u}'_j \right), \qquad \mathbf{r}^\star = R\mathbf{u}$$

$\mathcal{P}$ randomly chooses $\hat{\boldsymbol{\omega}}, \boldsymbol{\omega'} \in \mathbb{Z}_q^N$, $\omega_1, \omega_2, \omega_3 \in \mathbb{Z}_q$, and $\boldsymbol{\omega_4} \in \mathbb{Z}_q^w$, and sends the following values to $\mathcal{V}$:

$$\hat{c}_0 = h_1, \;\; \hat{c}_i = h^{\hat{\mathbf{r}}_i} \hat{c}_{i-1}^{\mathbf{u}'_i} \;\; (i \in \{1, \ldots, N\}) \qquad t_1 = h^{\omega_1} \qquad t_2 = h^{\omega_2} \qquad t_3 = h^{\omega_3} \prod_{i=1}^{N} h_i^{\omega'_i}$$

$$\mathbf{t}_4 = \mathbf{ReEnc}_{g,pk}(\textstyle\prod_{i=1}^{N} \mathbf{e}_i^{\prime \boldsymbol{\omega}'_i}, -\boldsymbol{\omega}_4) \qquad \hat{\mathbf{t}}_i = h^{\hat{\boldsymbol{\omega}}_i} \hat{c}_{i-1}^{\boldsymbol{\omega}'_i} \;\; (i \in \{1, \ldots, N\})$$

3 $\mathcal{V}$ chooses a challenge $c \in \mathbb{Z}_q$ at random and sends it to $\mathcal{P}$.

4 $\mathcal{P}$ then responds with

$$s_1 = \omega_1 + c \cdot \bar{r} \qquad s_2 = \omega_2 + c \cdot r^\diamond \qquad s_3 = \omega_3 + c \cdot \tilde{r} \qquad \mathbf{s}_4 = \boldsymbol{\omega}_4 + c \cdot \mathbf{r}^\star$$

$$\hat{\mathbf{s}} = \hat{\boldsymbol{\omega}} + c \cdot \hat{\mathbf{r}} \qquad \mathbf{s}' = \boldsymbol{\omega'} + c \cdot \mathbf{u}'$$

5 $\mathcal{V}$ accepts if and only if

$$t_1 = (\textstyle\prod_{i=1}^{N} \mathbf{c}_i / \prod_{i=1}^{N} h_i)^{-c} h^{s_1} \qquad t_2 = (\hat{c}_N / h_1^{\prod_{i=1}^{N} \mathbf{u}_i})^{-c} h^{s_2} \qquad t_3 = (\textstyle\prod_{i=1}^{N} \mathbf{c}_i^{\mathbf{u}_i})^{-c} h^{s_3} \prod_{i=1}^{N} h_i^{\mathbf{s}'_i}$$

$$\mathbf{t}_4 = \mathbf{ReEnc}_{g,pk}((\textstyle\prod_{i=1}^{N} \mathbf{e}_i^{\mathbf{u}_i})^{-c} \prod_{i=1}^{N} \mathbf{e}_i^{\prime \mathbf{s}'_i}, -\mathbf{s}_4) \qquad \hat{\mathbf{t}}_i = \hat{c}_i^{-c} h^{\hat{\mathbf{s}}_i} \hat{c}_{i-1}^{\mathbf{s}'_i}$$

- A matrix $M$ is a *permutation matrix*, if it contains only 0 and 1 values and, moreover, if every column and every row contains exactly one 1.
- $M\mathbf{x}$, for $M \in \mathbb{Z}_q^{N\times N}$ and $\mathbf{x} \in \mathbb{Z}_q^N$, is a vector of length $N$ where $i$-th position is equal to $\sum_{j=1}^{N} M_{i,j}\,\mathbf{x}_j$.
- For a permutation $\pi$ of the set $\{1,\ldots,N\}$, we will denote by $M_\pi$ the *permutation matrix defined by* $\pi$ which is the permutation matrix such that, for all $i \in \{1,\ldots,N\}$, value 1 in the $i$-th column is at position $\pi(i)$. It follows that, if $\mathbf{y} = M_\pi\mathbf{x}$, then we have $\mathbf{x} = (\mathbf{y}_{\pi(1)},\ldots,\mathbf{y}_{\pi(N)})$.
- $EPC_{h,h_1,\ldots,h_N}(\mathbf{m}, r)$, for $\mathbf{m} \in \mathbb{Z}_q^N$ and $r \in \mathbb{Z}_q$, is defined as $h^r \prod_{i=1}^{N} h_i^{\mathbf{m}_i}$ (otherwise known as an extended Pedersen commitment).
- $Com(M, \mathbf{r})$, for $M \in \mathbb{Z}_q^{N\times N}$ and $\mathbf{r} \in \mathbb{Z}_q^N$, is $(\mathbf{c}_1, ..., \mathbf{c}_n)$ where $\mathbf{c}_i = h^{\mathbf{r}_i} \prod_{j=1}^{N} h_j^{M_{i,j}}$, which means that $\mathbf{c}_i$ is the extended Pedersen commitment to the $i$-th column of $M$.
- A *ciphertext* is a pair $(x, y) \in G_q$. Ciphertexts can be multiplied component-wise, that is $(x, y)(x', y') = (xx', yy')$. One can also compute component-wise the $z$-th power of a ciphertext, that is $(x, y)^z = (x^z, y^z)$.
- $\mathbf{ReEnc}_{g,pk}(e, r)$, for $e = (x, y) \in G^2$ and $r \in \mathbb{Z}_q$ is $(xg^r, y\,pk^r)$.
- $\mathbf{ReEnc}_{g,pk}(\mathbf{e}, \mathbf{r})$, for $\mathbf{e} \in (G^2)^w$ and $\mathbf{r} \in \mathbb{Z}_q^w$, denotes the sequence $\mathbf{ReEnc}_{g,pk}(\mathbf{e}_1, \mathbf{r}_1), \ldots, \mathbf{ReEnc}_{g,pk}(\mathbf{e}_w, \mathbf{r}_w)$.
- $\langle \mathbf{a}, \mathbf{b} \rangle$, for $\mathbf{a} \in \mathbb{Z}_q^N$ and $\mathbf{b} \in \mathbb{Z}_q^N$ is $\sum \mathbf{a}_i\mathbf{b}_i \mod q$.
- $a\mathbf{x}$, for $a \in \mathbb{Z}_q$ and $\mathbf{x} \in \mathbb{Z}_q^N$, is a vector of length $N$ where $i$-th position is equal to $a\mathbf{x}_i$.
- For two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_q^N$ we write $\mathbf{x} + \mathbf{y}$, to denote the pairwise addition.

Note that, in Step 5, $\mathbf{e}_i$ is a multi-ciphertext (that is an element of $(G^2)^w$) and $\mathbf{u}_i$ is a number in $\mathbb{Z}_q$; the exponentiation is, therefore, applied component-wise.

This interactive algorithm is in fact a correct zero-knowledge protocol. For completeness, we provide the precise security statement as well as the proof in our technical report [5].

We will now present the non-interactive version of this zero-knowledge proof which is used in our e-voting system. This non-interactive version is derived from Algorithm 9 in the standard way using the Fiat-Shamir heuristic.

The non-interactive procedure for zero-knowledge proof generation is given in Algorithm 10. As one can see, this procedure follows the steps of the prover from Algorithm 9, where the challenges are derived from the proof statement and the protocol transcript (up to the given point) using a hash function.

**Algorithm 10:** ZK-Proof of Shuffle Generation

**Public Setup :**

- group $G$ of order $q$ with a generator $g \in G$, $pk \in G$,
- multi-commitment key $h, h_1, \ldots, h_N \in G$

**Input :**

- lists of input multi-ciphertexts $\mathbf{e}_1, ..., \mathbf{e}_N$ and output multi-ciphertexts $\mathbf{e}'_1, ..., \mathbf{e}'_N$, where $\mathbf{e}_i, \mathbf{e}'_i \in (G^2)^w$,
- permutation $\pi$ of the set $\{1, \ldots, N\}$,
- random coins $\mathbf{r} \in \mathbb{Z}_q^N$ and $R \in \mathbb{Z}_q^{N \times w}$ (as in Algorithm 9),

1 Derive a challenge $\mathbf{u}_i \in \mathbb{Z}_q$ (for $i \in \{1, \ldots, N\}$) from the following values (in the given order):

$$g,\ pk,\ h,\ h_1, \ldots, h_N,\ \mathbf{e}_1, \ldots, \mathbf{e}_N,\ \mathbf{e}'_1, \ldots, \mathbf{e}'_N,\ \mathbf{c},\ i$$

using Algorithm 4 (uniform hash) with parameter '$q$' set to $q$ (the group order), where $\mathbf{c} \in G^N$ is the commitment to the permutation matrix with randomness $\mathbf{r}$, that is $\mathbf{c} = \mathit{Com}(M_\pi, \mathbf{r})$

2 Carry out Step 2 of Algorithm 9, defining $\hat{c}$ as $\hat{c}_1, \ldots, \hat{c}_n$ (notice that $\hat{c}$ does not include $\hat{c}_0$ which is simply defined as $h_1$)).

3 Derive a challenge $c$ from the following values (in the given order):

$$g,\ pk,\ h,\ h_1, \ldots, h_N,\ \mathbf{e}_1, \ldots, \mathbf{e}_N,\ \mathbf{e}'_1, \ldots, \mathbf{e}'_N,\ \mathbf{c},\ \hat{c},\ t_1,\ t_2,\ t_3,\ \mathbf{t}_4, \hat{\mathbf{t}}$$

using Algorithm 4 (uniform hash) with parameter '$q$' set to $q$ (the group order).

4 Carry out Step 4 of Algorithm 9.

5 Output the ZK-proof consisting of ($\mathbf{c}$, $\hat{c}$, $t_1$, $t_2$, $t_3$, $\mathbf{t}_4$, $\hat{\mathbf{t}}$, $s_1$, $s_2$, $s_3$, $\mathbf{s}_4$, $\hat{\mathbf{s}}$, $\mathbf{s}'$).

The corresponding non-interactive procedure for zero-knowledge proof verification is given in Algorithm 11. This algorithm derives the challenges in the same way as it is done in Algorithm 10 and checks the equation of the last step of Algorithm 9.

We emphasise that, for all the shuffle proofs generated for an election, the fixed, verifiably generated multi-commitment key is used, as specified earlier in this section. Therefore, for shuffle proofs generated for election instances specified in this document, **the verification algorithm should use this fixed multi-commitment key**.

**Example:** We now present an instance of a valid proof of shuffle for the group secp256k1 and the standard fixed multi-commitment key (as defined earlier in this section). The used public key is:

$$pk = \texttt{030000000000000000000000003B78CE563F89A0ED9414F5AA28AD0D96D6795F9C63}$$

The sequence $\mathbf{e}_1, ..., \mathbf{e}_N$ of input multi-ciphertexts with $N = 5$ and $w = 3$ is given in the JSON format:

```
[ {
  "ciphertexts" : [ {
    "x" : "0214C8ED687A03590A5DFF16207636B75641E4265077C0C8546FF820188662EB5F",
    "y" : "03E08BA84715A5CC14B27895F7A42194933CF7D9B71101F99CDEB0800399F02C20"
  }, {
    "x" : "032392DF7D3B7BDFFF9A7ACC599E20D4C435755B4EF73C458900B0928B34864DAE",
    "y" : "02234DBE74A1D90B6D7EE8A376958347028B07A71DEB69FD203692AA9CCE86865A"
  }, {
    "x" : "02B4EA4738B32421826F1F87A4712372059F4DEDBA136155BAAEA76FDA77FD1B77",
    "y" : "03577F6FEFC44A662ACA61A09A3C0D6D57B5BBAE5D54DEC8843F1A3CBC5096B557"
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "0203B5E69A3B368C16351D2AFB94135A13C3E3E325F0D04612F31FC8E79CA8AF23",
    "y" : "020833D683DC989F9F43CC5C6229A01D8036E4B0EF980E9B3A5035D4267E85B328"
  }, {
    "x" : "02305032260DE32A5C1C2E38AB18229AD1A52540A1333E265EEDA3060BC84EDD10",
    "y" : "024FC453BDF7159175F8A647DDD81E124A9BC988AFDEAE4F578A0A0F1C7AFD01C4"
  }, {
    "x" : "029EE3273853C61A09EE816D93220D3BB268E843B84516DB3E19108FEC719E9B29",
    "y" : "036AB3656C933BF8A6C07A1031FBB502250CB966A25BA1A0E668718D20DCC594CC"
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "020D5725F57B7B90DC5D16A45C121F8B8ACF2981490E5BF7756F78E1FFC9010364",
    "y" : "0338E08940BC3F3681DB354930ACEB98C4EBB7C7796225DD274D5E9B828A18FDC8"
  }, {
    "x" : "03AE8B4B0DC1A0DCE73DC5BE8F8F45B76D349FA77A22109D0647890B035DF1C18D",
    "y" : "025718470F1BE2A75653CD608CF2999E5A1F7B57778C76DB5B74AED2084C87B624"
  }, {
    "x" : "02811B7C71E2C5EC448FB9E937FB97501E2DB95B080C0FC3639EC25FBE06BB859F",
    "y" : "038CCFC46E348B0A2D5FF4DB1EFA4E4388D706D133B13B35A1D0D582B50A2B4495"
  } ]
}, {
  "ciphertexts" : [ {
```

**Algorithm 11:** Verification of a ZK-proof of Shuffle

**Public Setup :**

- group $G$ of order $q$ with a generator $g \in G$, $pk \in G$,
- multi-commitment key $h, h_1, \ldots, h_N \in G$

**Input :**

- lists of input multi-ciphertexts $\mathbf{e}_1, ..., \mathbf{e}_N$ and ouput multi-cipherexts $\mathbf{e}'_1, ..., \mathbf{e}'_N$, where $\mathbf{e}_i, \mathbf{e}'_i \in (G^2)^w$,
- the ZK-proof $(\mathbf{c}, \hat{c}, t_1, t_2, t_3, \mathbf{t}_4, \hat{\mathbf{t}}, s_1, s_2, s_3, \mathbf{s}_4, \hat{\mathbf{s}}, \mathbf{s}')$, where

$$\mathbf{c} \in G^N, \quad \hat{c} = (\hat{c}_1, \ldots \hat{c}_n) \in G^N,$$
$$t_1 \in G, \quad t_2 \in G, \quad t_3 \in G, \quad \mathbf{t}_4 \in (G^2)^w, \quad \hat{\mathbf{t}} \in G^N,$$
$$s_1 \in \mathbb{Z}_q, \quad s_2 \in \mathbb{Z}_q, \quad s_3 \in \mathbb{Z}_q, \quad \mathbf{s}_4 \in \mathbb{Z}_q^w, \quad \hat{\mathbf{s}} \in \mathbb{Z}_q^N, \quad \mathbf{s}' \in \mathbb{Z}_q^N,$$

Note that $\hat{c}$ does not include $\hat{c}_0$ which is defined as $h_1$.

1 Derive $\mathbf{u}_i \in \mathbb{Z}_q$ (for $i \in \{1, \ldots, N\}$) from the following values (in the given order):

$$g,\ pk,\ h,\ h_1, \ldots, h_N,\ \mathbf{e}_1, \ldots, \mathbf{e}_N,\ \mathbf{e}'_1, \ldots, \mathbf{e}'_N,\ \mathbf{c},\ i$$

using Algorithm 4 (uniform hash) with parameter '$q$' set to $q$ (the group order).

2 Derive a challenge $c$ from the following values (in the given order):

$$g,\ pk,\ h,\ h_1, \ldots, h_N,\ \mathbf{e}_1, \ldots, \mathbf{e}_N,\ \mathbf{e}'_1, \ldots, \mathbf{e}'_N,\ \mathbf{c},\ \hat{c},\ t_1,\ t_2,\ t_3,\ \mathbf{t}_4, \hat{\mathbf{t}}$$

using Algorithm 4 (uniform hash) with parameter '$q$' set to $q$ (the group order).

3 Carry out Step 5 of Algorithm 9. Output 'true' if and only if all the equations of this step hold true. Notice that $\hat{c}$ (given as the input to this algorithm) does not include $\hat{c}_0$ (a value necessary to compute $\hat{\mathbf{t}}$) which is simply defined as $h_1$.

```
    "x" : "02DC464302FBE1CE58ECE95B1A40B4FD99C190456844E17D62AA51801C246668F2",
    "y" : "033AC4F52C385066CBBBC04BDC87D87232F58B02B98F7F4661C766114FDD950A65"
  }, {
    "x" : "032C50FD2FB9811BC47A76D1478E13A3E5B4631AF8B97D3EB32347FB597FC0D72C",
    "y" : "02FFA0CFD2B1E7CA591BFA8E33AB9EB54005A773DF5E0A96371DF3ADCDDC58D965"
  }, {
    "x" : "023460FDA0AF0D22DFF6F18C2CDD7A8325EB9EA28C177D83CB2BB2CD7FE10ADE66",
    "y" : "03ED73B8D97CEED2B56924D91A1C48330DA0669CA2323463C4EC211429704E2C2D"
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "024622D7BD63EA32760775DEA4F70531445C22BBFD4E4BCDB9B65E93E728916DF1",
    "y" : "03D21229F0D579ACBDCBA52A69D452FFA15BE431AB44E053F8C045690643AFEEA4"
  }, {
    "x" : "0292B9CE0CEA817BDA811CD1B0687F18EEA410FF4AED585E57F2CB2772AEB12089",
    "y" : "03215C6F22892AF7AFF3D07D5945C33E0ECA2A71114E7735182E0A426925A3B943"
  }, {
    "x" : "039B21A650670E8C9266A8B9951CA667C417ABF10D872FC518295CEF251365CC51",
    "y" : "0341B2AE82C303AA11EB36F888C0EE84A3CB45EB3CC2EA2A9892453876E1A63732"
  } ]
} ]
```

The sequence $\mathbf{e}'_1, ..., \mathbf{e}'_N$ of output multi-ciphertexts is given in the JSON format:

```
[ {
  "ciphertexts" : [ {
    "x" : "02456DE1179DB1B71A99277A3DE50F4AABB8F067E919394094F1796975D61C8241",
    "y" : "0204B89C49C36D914A6345FC891B2D2C218CD6EC92618246D6CBCEDC8B3915E7B3"
  }, {
    "x" : "0209C4E8F415C4BD201783206EACBF2350D621DDB54AFBBE530FA4FA01618DF539",
    "y" : "02BA2F9D6BA01F64B36DE7BF8C0B6FFB54C3E4E8E3A208353827F415701C07E8F8"
  }, {
    "x" : "0369715046EA5CA8A5BB300ADDC7FA7DC46A7F3CDBB5F0E3F8DB91D018B8C9C973",
    "y" : "031ED897C05609D58C216BA04B7C59E914CCA2CF260F18E9C9E61831DEE9FBB47F"
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "020C1C7A58906A5E7FEA11D8499F3FC6807F8A9214E82D0448E7D1128BC7BA216F",
    "y" : "03BC78094A4296878A77FE026D689FD10FD5DE3005FF512BA19E599AB9EE51D46B"
  }, {
    "x" : "036CD0CC7159AE847A80BFCC26088F919EA808343CC631E6B7D17534AB9364A1C2",
    "y" : "036721AB124B5E691355E269ED85EC7225DF2F1E2689A3904E63CDB99557DE590E"
  }, {
    "x" : "02813373C27103E17FD43EBFBEF79CA7BA31F3F17E707B42362B7848063D10A271",
    "y" : "032C9A98CDEB02444DF56B09553C36FCBDB9B470B1A39518EAFD64018F35999262"
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "03AB7BE97B4A7B879B9F893C03AA7B477BF34785F80FC967EC75144882BB8EA72C",
    "y" : "02A641223FC94FEE1E0E7F24E1C1B475BA45F91364984D94F3FB529812C26EBC34"
  }, {
    "x" : "02F7340EAA4299973AB27231C27E53B90B913554F03721BA06E3C344DE72F42A0C",
    "y" : "02128077E42835DB0B2A992BA9A3415481C2EC1A9350C07674111CFFD3528348C6"
  }, {
    "x" : "0372B6FAD5684760BA7105A449112D78EF4F60CC80752B33CCB79852AFF36BEAC7",
    "y" : "02A284A3A53AE803E684F5E903DB687354F7C21CFA51C304691C1D5DE0394883F9"
```

```
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "0279045EEC54A7C574B0475932ACA3F7C5022DB5DA63CCF59486037E87FFF5FC13",
    "y" : "0339C7B1DAFF747CA0BB51199909C51EE1A2A8D85678CFFCC10834CA850F923BA5"
  }, {
    "x" : "0287DBE4AC3C675718332715FD10B8A65F75A79769F664B8C9EF7138BE27DCEC3D",
    "y" : "022F8D43DEA945BC03299F4285CDE8FF028EAA7352826490F1DE2C8A3E7200541F"
  }, {
    "x" : "03A06D4592A73B1211C0E7616C502CC1A851CE869F6474A24869B5D692D022BE83",
    "y" : "03F43A46AC5B017F8C090B1BB19FFBF91FC0BBD635980BE36028A07C9A78AE73CC"
  } ]
}, {
  "ciphertexts" : [ {
    "x" : "033BCD27079C93D3A0D5489EA4E97A37AD2C0BB993AB03828F15AFDCE86A09D1EE",
    "y" : "02E20CC62C67B3D7ACA45D35F0982ADB2AFE00B211491E2BEE57FF839E2AA77891"
  }, {
    "x" : "0227403C521B76EF97C257042F95B67EF0EBCF38CBEF6147ED384A696D09B3B4F1",
    "y" : "02055EF3DF1BB93420BAC356B7E4FFF281F996E06955A67ED8E7A81D0F2E902030"
  }, {
    "x" : "03B4BF902DF1EB8BF992E1FCB7DAFBFA52DF056869879A4EFA2EE617224B2028AA",
    "y" : "02C5945ED05DD243D456DBD883BE62062CE160B7947C1931B16E9A742A643CA9A7"
  } ]
} ]
```


The proof of shuffle is given by the following JSON. Note that the components of $t_4$ are given in separate lists (first components in `t4x` and second components in `t4y`).


```
{
  "c" : [
    "03063CA66F8C0ECEAAB8236BA467F3D817710FBF45792A6AC31DF6AF4293F3F5FC",
    "038B87D18C31424A6C217FF70AA58F5B5682093DE6BCC7073E66DAE1BF6B32BE0D",
    "038C81504B353A3DA542AB71B2D9F303053305E675C2D6D09B59692C72C79BC6C7",
    "03EBE3705F74AF1E0E0E5CEF5C55A0C4768EF94B656799CF81A7D3A7664F01A1E7",
    "03CCAB079B02DFED7DB56621060FBF2A795E39D4CDEBA4A9B7799AB92191484DA5"
  ],
  "cHat" : [
    "02015DCE2FFFFCA395A6E62C0F2661BF4F4D17AEDBAD90B1CD3C111B7E9FEF0DCE",
    "03560966D35CF68F89F81E233D329F1C64E8F5991D270CEF49843BD00E6DC5CBF8",
    "027718D13F8E0AC62E269507BB3FBDACDDFF11D29E6971C64F2BFF9535735FF5CE",
    "033749E1628EEB14EE6AD370BD47F430020A3E7CD77A3D2CF20F5177EE79FDA462",
    "03EE78C0CD5E0320E8BC342C420D873802ED1FF2AE8D9D4FC165B405821394E82C"
  ],
  "t" : {
    "t1" : "035C091CBE9D77D0030FE3584CC20DF2C1DB0621F07E3C16600981C9A806C39E15",
    "t2" : "0211DA61A861A0B7AF9D3A77022EB55B511D4B07F18D271B6C61ADFF155BD419B1",
    "t3" : "023D82EA9E9ADC647E699EF1901BCBB89B7D587DAF328BA354B100020245C1E6E7",
    "t4y" : [
      "03919EDA2E6275ABA58260D4B0A260E8E6A2D9BB99486CFB1BAB41C56400FFF473",
      "039681898C617F1D9304B1B3103C009CDF98B6509A768C89B3DC1D6D97102414AD",
      "031E12721D6AB8E4789ABD2EB074BCBEE4B931112B23F45C5CBBA2040DF0CDBAFE"
    ],
    "t4x" : [
      "03C0CE5B3CB78C48072687ADB95A74928065C5E02197E8FA84CB82FFF5F32B56C2",
      "030DE4191A9F916DDA2BC0F89B6CF91FB86504A937116F0F5390F947CB19C9B2DF",
      "022204EADACC736A723A774F2F852DACE0FE2063A35B6DC73DB02CEE9C6913338E"
```

```
    ],
    "tHat" : [
      "02A6B5848F4C8A548DEC32CF69DD62102A0229A91754833011ABEDF7C37CAD0B35",
      "03AFFAFC3E79F3F0D5E167D28CD07E32FB2A2013E3C4E10228A1CF93EC01191D24",
      "02D4CC74684CAC1AAF3B097202A53D6C95E56063A9FAACFB83880A9275ECF1DB5F",
      "0247C0DAF54B4BBB57CD02055F669B6C3AF8A2772B7A4D724B028F80CC85F7905F",
      "02BD163CFCF9F8F5220D97C2CAEED5ED6F8631FA440EDFE62EF42A1130A10D60BF"
    ]
  },
  "s" : {
    "s1" : "691403168808870084292994334095347926968484190932168736805519748971448871596 10",
    "s2" : "531311801158194716036496618858378489518678929706034918947782426823012548034 39",
    "s3" : "665458857868125362074344322226404779956310706434291603791212997678285299230 58",
    "s4" : [
      "492341498692505949593015148069278807616215946523829298649165364785672935764 03",
      "441193659166381032297790401462562314835381643071183629443588528578211165056 72",
      "864620110212562475105873567064380165955271689713539302546071869658332380876 71"
    ],
    "sHat" : [
      "875795230461220189878174010094612041211602460148606001189013588822592842856 35",
      "563146904573362360858578316798248944267389819046947251284595207672015103023 00",
      "257536798534188791908968909322706653316403580300576555504575090947276625951 42",
      "639151295138659111365976043406499088300393435819132114366249024296433583482 25",
      "698543450230464567523509154837940018459938656281809226591934970653287280763 15"
    ],
    "sPrime" : [
      "723662745720445281026699831910110329539914845880991217692466448825829088865 23",
      "173526105758966639214313353323165188051528846817692389351432880478731481083 74",
      "565956895154257803221671500599262699897573261875038883918507559589395835066 50",
      "932080007410437411838489719081959218615812557313705042807443409080219225173 10",
      "143571025506288364720115210080200571341712593730418929812999232067530764241 52"
    ]
  }
}
```


The values of the challenges $u_1, \ldots, u_5$ and $c$ computed by the verification procedure (Step 1 and 2) are:

$$u_1 = 25423173261403838045780498659101929143374212024885961749677191123894345457203$$

$$u_2 = 10716339517378055212095983968273491541929410843531543488432949344929770251698 9$$

$$u_3 = 27527278399601879823598941265103560004203827776425685772980663219237052390097$$

$$u_4 = 10638210745354102438351977402204811945276838644434185138171916009292618733984 7$$

$$u_5 = 78025421855638301792439005550141533632218318123084187717794732643161239341502$$

$$c = 14886957920142020425415970750713297044432709962075734803391029210025459699280$$

## A.7 Key Generation

In this section, we are concerned with the simple key generation process, where the election key is generated by one authority (POLYAS). The variant of our election system with key sharing and threshold decryption is not covered by this document.

The secret (decryption) election key and the corresponding public (encryption) election key are generated by the Election Provider (Polyas). The secret key $s$ is a random number in $\mathbb{Z}_q$. The corresponding public election key $pk$ is derived from the private key in the standard way:

$$pk = g^s$$

and published, along with a zero-knowledge proof $\pi$ of the knowledge of the corresponding secret key. The public election key used, in particular, by the voter client to encrypt the voter's ballot.

The zero-knowledge proof $\pi$ of knowledge of the secret key corresponding to the given public key $pk$ is the non-interactive variant of the well-known zero-knowledge proof of the knowledge of the discrete logarithm of $pk$ [10], where we use the strong Fiat-Shamir heuristic [2]. It is of the form $\pi = (c, f)$ and is created by drawing a random number $a$ from $\mathbb{Z}_q$ and computing

$$\begin{aligned} A &= g^a \\ c &= \mathit{UniformHash}_q(g, pk, A) \\ f &= a + cs \mod q \end{aligned}$$

where $\mathit{UniformHash}_q$ denotes the uniform hash function defined in Algorithm 4 with parameter '$q$' set to $q$ (the order of the group).

***Note.*** *A record containing the public election key along with the corresponding zero-knowledge proof is published on board keygen-electionKey-Polyas (Section E.6.2). Such a record is of the type PublicKeyWithZKP (Section F.32).*

The public output of the key generation process (the public election key with the mentioned zero-knowledge proof) should be checked using the verification procedure described in Algorithm 12.

**Algorithm 12:** Verification of the public election key with the ZK-proof

**Input:** The public election key $pk \in G$ and the zero-knowledge proof of the knowledge of the corresponding secret key consisting of $c \in \mathbb{Z}_q$ and $f \in \mathbb{Z}_q$

1 Compute $c' = \mathit{uniformHash}_q(g, pk, \frac{g^f}{pk^c})$, where $\mathit{uniformHash}_q$ denotes the uniform hash function defined in Algorithm 4 with parameter '$q$' set to $q$ (the order of the group). Note that the computed value is an element of $\mathbb{Z}_q$.

2 Check that $c = c'$ and reject the proof as incorrect if this is not the case.

**Example:** For the group `secp256k` and the public key

$pk$ = `03403091F3E81EE0E125FC33614DBA1ADBA569A3F7C05F9B36587054151508D490`

The following proof (in JSON format)


```
{
  "c" : "623279416854868254491349971992896696842074651475654801405326346508654722771 54",
  "f" : "519626871623585287094092584076364651783882473066691529650126972661198031185 83"
}
```


is a valid zero-knowledge proof of knowledge of the private key, for which Algorithm 12 succeeds.

## A.8 Verifiable Decryption

The ballots, after being shuffled in a verifiable way (Section A.6), are decrypted by the Election Provider (Polyas). This decryption step is also verifiable, which means that appropriate zero-knowledge proofs are produced, allowing an independent party to make sure that this step has been carried out correctly, without revealing the used private key.

***Note.*** *The decryption is applied to all the ciphertexts in all the output mix packets published on board mixing-mix-Polyas (Section E.7.2). For each such packet a corresponding packet with decrypted messages is published on board decryption-decrypt-Polyas (Section E.4.1), as explaned below.*

An encrypted ballot is represented as a *multi-ciphertext*, i.e. a sequence $e = e_1, \ldots e_n$ of simple ciphertexts, where every ciphertext $e_i$ is of the form $(x_i, y_i)$ with $x_i, y_i \in G$.

In the first step of the decryption process, for each $i \in \{1, \ldots, n\}$, a so-called decryption share $\alpha_i = x_i^{sk}$ is computed, where $sk$ is the secret election key. From this value the decryption result $d_i \in [0, \mathit{messageUpperBound})$ can be easily computed: $d_i = \mathit{decode}(y_i/\alpha_i)$, where *decode* denotes the decoding function, which for a given group element returns an element in $[0, \mathit{messageUpperBound})$, as defined in Section A.2.1.

The values $d_1, \ldots, d_n$ are then combined and decoded into a binary message (a byte array) using Algorithm 6, where the parameter '$q$' is set to *messageUpperBound*. This message is the result of the decryption.

For this result, the following *zero-knowledge proof of correct decryption* is constructed. For every $i \in \{1, \ldots, n\}$, a zero-knowledge proof $\pi_i$ is produced which shows that the $i$-th decryption share $\alpha_i$ is valid. This proof is a non-interactive variant of the well-known zero-knowledge proof of knowledge of discrete logarithm equality [3], where we use the strong Fiat-Shamir heuristic [2]. This proof is of the form $\pi_i = (c_i, f_i)$, where

$c_i, f_i$ are generated by drawing a random number $a_i$ from the set $\mathbb{Z}_q$ and computing

$$
\begin{aligned}
A &= g^{a_i} \\
B &= x_i^{a_i} \\
c_i &= \mathit{uniformHash}_q(g, x_i, pk, \alpha, A, B) \\
f_i &= a_i + c_i \cdot sk \mod q
\end{aligned}
$$

Note that, indeed, $(c_i, f_i)$ is a non-interactive zero-knowledge proof of equality of discrete logarithms $\log_g(pk)$ and $\log_{x_i}(\alpha_i)$ (which both are equal to $sk$). The overall proof of correct decryption for the message above consists of the all the decryption shares and the corresponding proofs: $(\alpha_1, \pi_1, \ldots, \alpha_n, \pi_n)$.

***Note.*** *The output of this step is published on board decryption-decrypt-Polyas (Section E.4.1), where the decrypted binary messages along with zero-knowledge proofs of correct decryption are organized into records of type MessageWithProof (Section F.27) and then aggregated in packets of type MessageWithProofPacket (Section F.28) and published on board decryption-decrypt-Polyas (Section E.4.1).*

In order to verify the correctness of the decryption operation one can use the procedure presented in Algorithm 13 (applying this algorithm to every decrypted message with a proof and the corresponding input multi-ciphertext $e$).

**Example:** In this example, we use, as before, the group `secp256k` as defined in Section A.2 and represent elliptic points in the hexadecimal representation of the compressed form of ANSI X9.62 4.3.6. For the public key

$$pk = \texttt{03403091F3E81EE0E125FC33614DBA1ADBA569A3F7C05F9B36587054151508D490}$$

and the multi-ciphertext

```
{
    "ciphertexts":[
        {"x" : "03B467D17D26AE0A29034C698A15F8E50C7DD17D43F2F088091479B8C0B9CFFC60",
         "y" : "037EA63D9BB3E1FE8AB74DB33E60DCA353878B9169D01585D53F59A30DB7029C16"}
    ]
}
```

the following record in JSON format is a valid decryption record for which the verification procedure should succeed.

```
{
    "message":"01060102020401000000000000000000",
    "proof":[
        {
            "decryptionShare" : "0220BF3495CDBDFCE2D10EB330AB72A56FC67B7D12BDB9270BB18D896089787FC6",
            "eqlogZKP" : {
```

**Algorithm 13:** Verification of ballot decryption

**Input:**

- public parameters: parameters of the used group $G$ (including the generator $g$ and the group order $q$), and the public key $pk \in G$,
- a multi-ciphertext $e_1, \ldots, e_n$, where $e_i = (x_i, y_i) \in G^2$,
- decryption result *message* given as a binary message (byte array) and a zero-knowledge proof of correct decryption $(\alpha_1, \pi_1, \ldots, \alpha_n, \pi_n)$ with $\alpha_i \in G$ and $\pi_i = (c_i, f_i) \in \mathbb{Z}_q^2$.

1 Check that all the components of the input are in the expected domains.

2 **for** $i$ *in* $1, \ldots, n$ **do**

3 Compute $A_i = \frac{g^{f_i}}{pk^{c_i}}$ and $B_i = \frac{{x_i}^{f_i}}{{\alpha_i}^{c_i}}$.

4 Compute $c'_i = \mathit{uniformHash}_q(g, x_i, pk, \alpha_i, A_i, B_i)$, where *uniformHash* denotes the uniform hash function defined in Algorithm 4.

5 Check that $c'_i = c_i$ and reject the record as incorrect if this is not the case (this check concludes verification the proof of discrete logarithm equality for bases $g, x_i$, and values $pk, \alpha_i$).

6 Compute, for each $i \in \{1, \ldots, n\}$, $d_i = \mathit{decode}(y_i/\alpha_i)$, where *decode* denotes the decoding function which for a given group element returns an element in $[0, \mathit{messageUpperBound})$, as defined in Section A.2.1.

7 Combine the values $d_1, \ldots, d_n$ into a binary message using Algorithm 6, where the parameter '$q$' is set to *messageUpperBound*, and check that it yields the message *message*; reject the record as incorrect if this is not the case (this check concludes that the decryption shares have been correctly combined into the final message).

```
            "c" : "868559846572463420252616817490333044376876919350408254900162470579420469772 71",
            "f" : "977490878123748274575683183135506792776051948271702216992646207926576238306 05"
          }
        }
    ]
}
```


## A.9 Final Ballot Tallying

In this section we describe how decrypted ballots in a binary format are decoded and interpreted and then how the final result is computed.

An encoded ballot may, in the general case, contain encoded voter's choices for more than one ballot sheet (the voter may be eligible to vote for one or more ballot sheets). The list of these ballot sheets is specified by the *public label* associated with the ballot (as we will see below, encoded ballots are grouped in packets sharing the same public label and, hence, the same list of ballot sheets). More precisely, a public label is a concatenation of ballot sheet identifiers, where the `":"` character is used as a separator. For instance, public label `"a:b:c3"` represents the list of ballot identifiers `a`, `b`, `c3` (the order is significant); therefore an encoded ballot with such a label is supposed to encode voter's choices for these three ballot sheets.

***Note.*** *To decode an encoded ballot, one needs to know the exact structure the ballot sheets (with the identifiers specified by the public label). The Structure of ballot sheets is published on board registry (Section E.1.2), as a record of type EpRegistry (Section F.21). Such a record contains, under the field* `ballotStructures`*, a list of records of type Core3Ballot (Section* **??***), each of which specifies one ballot sheet. A record of this type contains, amongst the others, the field* `id` *with the ballot identifier, referred to by a public label.*

***Note.*** *Decrypted ballots in binary format (represented as hexadecimal strings are published on board decryption-decrypt-Polyas (Section E.4.1), where decrypted messages (along with additional proofs of correct decryption) are grouped in records of type MessageWithProofPacket (Section F.28). Every such a record includes a public label (which specifies the list of ballot sheet identifiers, where evey ballot included in this recored encodes choices for these ballot sheets) and a list of records of type MessageWithProof (Section F.27), where field 'message' contains the decrypted ballot. All these decrypted ballots, together with the public label, constitute the input to the final counting.*

**Structure of a ballot sheet.** The structure of a ballot sheet, defined by a record of type Core3Ballot (Section **??**), is the following. A ballot sheet contains, in particular, a list of records of type CandidateList (Section **??**). Each such a record contains a list of records of type CandidateSpec (Section **??**) representing candidates (voting options). These records describe how a ballot should be presented to the voter (title of the ballot,

texts to be displayed, number of check boxes next to different voting options, etc.), as well as validation rules, for instance, the minimum and maximum number of votes a voter can select for a given candidate or a given candidate list. Note that if an optional field (such as 'CandidateList.maxVotesTotal') is not present (or defined as 'null') the corresponding constraint does not apply.

A voter can mark the whole ballot sheet as invalid (if this is allowed by the ballot specification; see field `showInvalidOption`). A voter can select chosen candidates/voting options (from one or more candidate lists). In the case of list voting, a voter can also give her vote for a whole list. The ballot sheet specification specifies how many options can be selected and if list voting is allowed; see ballot validation below.

**Ballot encoding and decoding.** To encode voter's choice as a byte array, the following simple method is used. Starting with the empty byte array $B$, we first append to $B$ one byte indicating if the voter marked the ballot as invalid (1) or not (0). Then we iterate over consecutive candidate lists (the order is significant), and for each candidate list:

- we append to $B$ one byte containing the number of votes for the whole candidate lists (0 if list voting is not allowed)
- for every consecutive candidate on this candidate lists, we append to $B$ one byte containing the number of votes for this candidate.

If a voter can vote for more than one ballot sheet, we simply concatenate the encodings for consecutive ballot sheets (again, the order is significant).

The decoding procedure recovers the number of votes for different voting options (whole candidate lists and individual candidates), as well as the flags indicating which ballots are marked as invalid in a straightforward way.

**Example.** Consider a voter who can vote for two ballot sheets $A$ and $B$, where ballot sheet $A$ contains two candidate lists, each with two candidates, while ballot sheet $B$ contains only one candidate sheet with three candidates. Then, the sequence of bytes

$$0, 0, 2, 3, 0, 4, 5, 1, 0, 7, 7, 7$$

encodes the following voter's choice:

- The choice for the first ballot sheet (ballot sheet $A$) is encoded by the bytes 0, 0, 2, 3, 0, 4, 5 and means that

  - the ballot is not marked as invalid (first 0),
  - the voter gives 0 votes for the first candidate list and the candidates on this list get, respectively, 2 and 3 votes,
  - the voter gives 0 votes for the second candidate list and the candidates on this list get, respectively, 4 and 5 votes.
- The choice for the second ballot sheet (ballot sheet *B*) is encoded by the bytes 1, 0, 7, 7, 7 and means that
  - the ballot is marked as invalid (first 1),
  - the voter gives 0 votes for the first (and the only) candidate list and the candidates on this list get 7 votes each.

A binary encoded voter's choice which cannot be decoded (is too long short/too long, has wrong value of the 'mark as invalid' field) is discarded as malformed. In such a case the whole voter's encoded ballot is discarded (we do not try to decode ballots partially).

**Validation rules.** Once a ballot is successfully decoded, each decoded ballot sheet needs to be validated against the rules expressed in the ballot specification. Consequently, only valid ballot sheets from well-formed ballots contribute to the final result. Note that, if a ballot sheet is invalid, we discard only this ballot sheet (not the whole ballot). For instance, a voter can cast a ballot with two ballot sheets, where one is valid and one is not (it is intentionally marked as invalid or violates some validation rules)

To validate a decoded ballot sheet, we consult the corresponding ballot specification. A decoded ballot sheet is discarded as invalid, if one of the following cases holds:

V1. The ballot sheet is explicitly marked as invalid.

V2. One or more *candidate rules* are violated, that is for some candidate the number of crosses for this candidate is bigger or smaller than required by the corresponding candidate specification; see CandidateSpec (Section **??**) fields `maxVotes` and `minVotes`.

V3. One or more *candidate list rules* are violated (see CandidateList (Section **??**). fields `maxVotesOnList`, `minVotesOnList`, `maxVotesForList`, `minVotesForList`), that is:

  (a) the total number of votes given to candidates on this list (not including the votes for the whole list) is not in the allowed range from `minVotesOnList` to `maxVotesOnList`,

(b) the number of votes for the whole list (in the case of list voting) is not in the allowed range from `minVotesForList` to `maxVotesForList`.

V4. A *ballot sheet rule* is violated (see Core3Ballot (Section **??**), fields `minVotes` and `maxVotes`), that is the total number of votes (of all kinds) on the whole ballot sheet is not in the allowed range from `minVotes` to `maxVotes`.

**Final counting.** All the valid sheets, that is sheets from well-formed ballots which have not been not rejected according to the above rules, take part in the final counting. The procedure simply sums up all the votes for every candidate identifier (and/or candidate list identifier, in the case of list voting) included in those sheets. Note that, in the ballot specification, candidate identifiers are mapped to human-readable descriptions, which allows one to produce a final result in a readable form.

# B Cast-as-Intended with a Second Device

The Polyas Core 3 Verifiable e-voting system offers an option of the cast-as-intended verifiability, where voters can use a second-device, such as a mobile phone, to audit their ballots. The ballot audit process, as described in this section, is an instantiation of the method presented in [9, 8], with an optimisation presented below.

## B.1 High level description of the process and information flow

The ballot audit application carries out the ballot audit process, displays the content of the ballot cast on the voter's behalf, and offers a signed receipt to download. The overall process of ballot audit works as follows.

1. **Initialization:** The second device application is started by the voter who scans the QR-code displayed by their main voting device. This QR-code contains the link to this app along with a (partially encrypted) payload.

2. **Login:** The voter is prompted to authenticate themself by providing a *time-based one-time password* (TOTP), displayed on the first device. This application passes this one-time password to the vote server, along with the voter identifier and some data from the QR code, as explained in more detail below. The response of the server (the *initial response*) contains public cryptographic parameters of the election and additional cryptographic material used in the following steps of the ballot audit protocol.

3. **Verification:** In the verification step, the application sends to the vote server a message containing a random challenge (which is an important part of the zero-knowledge protocol). The server response (the *final response*) contains cryptographic material which (combined with the content of the QR-code) allows the audit application to finalize the verification and extract the plaintext voter's choice from the encrypted ballot.

4. **Confirmation and receipt:** If all the above steps have been completed successfully, the application displays the choice included in the cast ballot (extracted in the previous step) and offers a receipt signed by the voting server for download.

We emphasise that the election secret key is *not* used during this process. We also note that the election system, when participating in this exchange, does not learn any new information. In particular, it does not learn how the voter voted.

## B.2 An Optimisation for Long Ballots

As mentioned in the introduction, the ballot audit process described in this document implements the method presented in [1], [2]. We augmented, however, this method with the following optimisation. This optimisation changes also the way ballots are encrypted (by the main voting device).

In [1], [2], the voting device sends $r^* = x + r$ to the audit device (via the QR-code); this value is then used to decode the voter's choice in the last step of the protocol. In the version of the method presented here, the QR code contains, instead, the *randomCoinSeed* which is used to generate a pseudo-random sequence *randomCoins* used to decode the voters choice (see Sections B.7 and B.8).

The motivation for this change is as follows. For log multi-cipher texts (ciphertexts consisting of multiple ElGamal ciphertexts), directly following the method of [1], [2] would result in QR codes containing many big integers, quickly making the QR-code excessively big and impossible to handle.

For our optimisation, we need to first make the following change in the ballot encryption protocol.

First, instead of selecting a random encryption coin $r$ and then computing $r^* = x + r$, the first device selects random $r^*$ and then computes $r$ such that $r^* = x + r$ (and uses r as the random encryption coin). Justification for this step is that the resulting distributions are identical (note that the computations are done here modulo $q$).

Second, with the above modification, we can now apply the described above optimisation: instead of randomly choosing a sequence $r^*$, we randomly choose a seed, and

generate $r^*$ from this seed. With this, we can include the seed (which is of constant length independently of the size of the cipher text) in the QR code.

## B.3 The Setup

When the election system is deployed and bootstrapped, it creates so-called **second-device public parameters** that is parameters of the voting process relevant for the ballot audit. These parameters include

- the *public election key pk* (used to encrypt ballots),
- the *receipt verification key* (used to check signatures on the ballot receipts issued by the voting system to the voter),
- the *content of the ballots* used in the election.

The e-voting system offers the **fingerprint** of these public parameters (technically it is ths SHA-512 hash of the JSON representation of the second-device public parameters). This fingerprint can be downloaded via the Election Admin Panel.

The second-device application should be pre-configured with this fingerprint. With this configuration, the second-device application should not accept any data which does not match this fingerprint (by issuing the fingerprint, the e-voting system commits to the content of the public parameters; by having this fingerprint pre-configured, the second-device makes sure that the data exchanged with the e-voting system uses public parameters which are committed to).

In the universal verification phase — after the election has been tallied and the verification package produced — it should be checked that the content of the verification package is consistent with the fingerprint that the second device application was pre-configured with. This process is supported by the universal verification tool (note that the verification package contains the second-device public parameters). This check can be done either by the auditors (in which case the party hosting the second device application provides the auditors with the used fingerprints) or directly by the party hosting the second device.

## B.4 Content of the QR code

The QR-code, displayed to the voter on the first device, contains

- *the voter's (public) identifier vid*,

- *a nonce* which is a random value generated for the voter by the election system and passed to the main voters device; it is used as part of the authentication data,
- *an encrypted payload encC* containing (when decrypted) the *randomCoinSeed*; this value is used in the later steps of the protocol (see Section B.7); see also Section B.2 for an explanation of its role,
- *an encrypted payload encD* containing (when decrypted) the linking token $\ell$ (an opening to the ballot reference *ref*, see (1) on page 33).

Both encrypted payloads are decrypted in the same way. For decryption, one needs a secret *comSeed* provided in the *initial response* (see below) by the election system. A symmetric AES encryption/decryption key *comKey* is derived from this value, by

$$comKey = kdfCounterMode(comSeed, 16, \text{‘’}, \text{‘’}),$$

where *kdfCounterMode* is the key derivation function defined in Algorithm 1. The encryption/decryption uses AES in the GCM mode (with IV of length 12 bytes coming first, followed by the encryption itself and, finally, the TAG of length 16 bytes).

An example content of the QR code is given in Section B.10.1.

## B.5 Login request and response

In the initial step (which does not require authentication), the application fetches, from the election server, the general election data, such as the title and the supported languages. The application displays the title, along with the voter identifier passed in the QR-code and prompts the voter to enter the current time-based one-time password (as displayed by the first voting device).

**Login request.** Once the voter provides the password and proceeds to the login, the application issues a *login request* to the voting service containing

- *voter's identifier vid*,
- *the nonce* included in the QR code,
- *the one-time password* provided by the voter,
- *the encrypted payload encD* taken from the QR-code (containing the reference coin $\ell$),
- *a challenge commitment c*, computed as follows:

sample random $e, r \in \mathbb{Z}_q$ and compute $c = h^r g^e$, where $h$ is the pre-defined Pedersen commitment key (note that $c$ is a Pedersen commitment to $e$ with the randomization factor $r$); the values $e$ and $r$ will be needed in the later steps of the protocol.

An example of a login request data is given in Section B.10.3.

**Initial response.** If the login data is correct, the election server replies with an *initial response* containing the following data:

- The *authentication token*, intended for authentication of the successive calls,
- Some *election metadata*, such as the election UUID, title, and the available languages,
- The *public label* of the voter (which specifies which ballots the voter is eligible to vote for; see Section A.5.1),
- The *second device public parameters*, provided as a JSON string, containing the public election key $pk$, the verification key *verificationKey* of the server (to check the acknowledgements), and the description of the *ballot sheets content* of the election (given as a list of values of type Core3Ballot, Section **??**),
- The *seed comSeed* to be used for decryption of the QR-code content,
- The *signature* of the election system on the voter's ballot (*signatureHex*),
- The (encrypted) *ballot*

  $$e_1 = (u_1, w_1), \ldots, e_n = (u_n, w_n),\ \pi_1, \ldots \pi_n,\ lab,\ ref,\ \sigma.$$

  as recorder in the ballot box (see Section A.5.2).
- The *initial part of the ZKP protocol exchange*, consisting of group elements

  $$X_1, \ldots, X_n,\ Y_1, \ldots, Y_n,\ A_1, \ldots, A_n,\ B_1, \ldots, B_n,$$

  where $n$ is the length of the multi-ciphertext (the number of simple ciphertexts in the encrypted voter's ballot). These values will be processed in the later validation step, as defined in Algorithm 15.

An example of a login request data is given in Section B.10.4.

## B.6 Checking the initial response

The ballot audit application can now decrypt the encrypted payloads *encC* and *encD* (from the QR-code), to retrieve *randomCoinSeed* and $\ell$, using *comSeed* included in the

initial response, as described in Section B.4.

Next, the consistency of the initial response should be checked, as follows. These checks make sure that the returned data is consistent with the pre-configured fingerprint, the returned ballot is uniquely associated with this voter's identifier, and that the returned signature is correct and can be later used to make sure that the ballot is included in the final tally.

- **Fingerprint consistency.** The data in the above response should match the pre-configured second device parameter fingerprint. To verify this match, the ballot audit application should hash the *second device public parameters* (recall that they are given in the login response as a JSON string). More precisely, this JSON representation should be encoded as a byte array, using the UTF-8 encoding, and hashed with the SHA-512 function. The result of this hashing, when represented as a hexadecimal string, should be equal to the pre-configured fingerprint.
  If this check fails, the application should abort the ballot audit process with an appropriate message.
- **Correct association to the voter identity**. The value *ref* in the ballot returned by the server should fit the linking label $\ell$, as defined by Equation (1) (note that *vid* is also provided in the QR-code).
  These check confirms that the audited ballot is uniquely associated with this voter's identifier (via *ref*).
  *Remark: In the current implementation, the reference included in the ballot is not ref itself, but a truncated Base-64 representation of a SHA-256 hash of ref.*
- **Correct signature.** The signature *signatureHex* is given as a byte array in a hexadecimal encoding. This signature should be checked using the *verificationKey* from the initial response. This key is expected to be an RSA public key in the *X509* encoding (note that a key in such encoding, when prepended with `-----BEGIN PUBLIC KEY-----` and followed with `-----END PUBLIC KEY-----`, would constitute a valid PEM file). In order to verify the signature, the application first needs to compute the sequence of bytes to be signed, as specified in Algorithm 14.

## B.7 Challenge and the Final Message

Once all the above steps have been carried out successfully, the ballot audit application sends to the voting server the *challenge request* including (the authentication token from the login response and) values $e$ and $r$ used to generate the challenge commitment $c$ (see Section B.5). Note that these values open the challenge commitment included in

**Algorithm 14:** Ballot as normalized bytestring

**Input** : Ballot $(e_1, \ldots e_n,\ \pi_1, \ldots \pi_{n'},\ \mathit{lab},\ \mathit{ref},\ \sigma)$, where $e_i = (x_i, y_i) \in G^2$, $\pi_i = (c_i, f_i) \in Z_q^2$, and $\sigma = (c, f) \in Z_q^2$

**Initialization** : Start with an empty byte array and append to it the following elements, where $\mathit{appendWithLen}(x)$ represents the action of appending the byte-array representation of $x$ prepended with 4-byte representation of its length and $\mathit{appendNum}(n)$ represents appending 4-byte representation of an integer $n$.

1 $\mathit{appendNum}(n)$, where $n$ is the length of the input multi-ciphertext
2 **for** $i \in 1..n$ **do**
3 $\quad \mathit{appendWithLen}(x_i)$
4 $\quad \mathit{appendWithLen}(y_i)$
5 $\mathit{appendWithLen}(\mathit{lab})$
6 $\mathit{appendWithLen}(\mathit{ref})$
7 $\mathit{appendNum}(n')$, where $n'$ is the length of the number of elements $\pi_i$
8 **for** $i \in 1..n'$ **do**
9 $\quad \mathit{appendWithLen}(c_i)$
10 $\quad \mathit{appendWithLen}(f_i)$
11 $\mathit{appendWithLen}(c)$
12 $\mathit{appendWithLen}(f)$

the login request.

If the provided values are valid (that is if they, in fact, constitute a valid opening to the commitment from the login request), the server responds with the final message $z = z_1, \ldots, z_n$, with $z_i \in Z_q$, of the zero-knowledge protocol exchange.

The examples of the challenge request and the corresponding response are given in Sections B.10.5 and B.10.6.

The ballot audit application, after having received the final message $z$, carries out the validation procedure defined in Algorithm 15, in order to determine, if the zero-knowledge proof exchange should be accepted. If any of these checks fails, the protocol is aborted. Note that the arithmetic operations are carried out in the underlying cyclic group (see Appendix A.2).

## B.8 Decoding and displaying the voter's choice

When the above steps have succeeded, the ballot audit application displays the plaintext voter's choice.

In the first step, the plaint-text voter's choice needs to be extracted from the encrypted ballot with the help of random coins $r_1^*, \ldots, r_n^*$ (see Algorithm 15, line 3). For this, we compute the sequence of group elements

$$c_i = \frac{w_i \cdot Y_i}{pk^{r_i^*}} \qquad \text{for } i \in 1 \ldots n,$$

where $w_i$ is the second component of the $i$-th ciphertext and $pk$ is the public election key.

We then map this sequence of the group elements (elliptic curve points) to a sequence of numbers in $Z_q$, using the decoding algorithm defined Appendix A.2.1. This sequence of numbers is then transformed to a byte array using the method described in Appendix A.9. Let us call this byte array the *encoded choice*.

Such an encoded choice can be now interpreted against the ballot definition, as provided in the login response. This ballot definition contains the content of the ballots, which allows the audit device to display the voter's choice in an explicit form, similar to how it looked in the first voting device. Note that the public label of the voter (also returned in the initial response) is relevant for this step, as it defines the list of ballots (for multi-ballot election) the voter is eligible to vote for. The details of the interpretation of the encoded choice against the ballot definition are given in Appendix A.9.

**Algorithm 15:** Final ZKP validation

**Input:**

- the initial message (part of the initial response) $X_1, \ldots, X_n$, $Y_1, \ldots, Y_n$ $A_1, \ldots, A_n$, $B_1, \ldots, B_n$, (all belonging to the group $G$),
- the ciphertexts $(u_1, w_1), \ldots, (u_n, w_n)$,
- the challenge e,
- the final message $z = z_1, \ldots, z_n$, with $z_i \in Z_q$,
- the public election key $pk$.
- *randomCoinSeed* (obtained, as described above, by decrypting *encC*, see Section B.4),

1 Check that $A$, $B$, $X$, $Y$, and $z$ are of the same length as the length $n$ of the ciphertext.

2 For all $i \in 1..n$, check equalities

$$A_i = \frac{g^{z_i}}{X_i^e} \quad \text{and} \quad B_i = \frac{pk^{z_i}}{Y_i^e},$$

3 Compute the sequence of random coins

$$r_1^*, \ldots, r_n^* = \mathit{numbersFromSeed}(n, \mathit{groupOrder}, \mathit{randomCoinSeed}),$$

where function *numbersFromSeed* is defined in Algorithm 3.

4 For all $i \in 1...n$, check that

$$u_i \cdot X_i = g^{r_i^*},$$

where $u_i$ is the first component of the $i$-th ciphertext, $X_i$ is as above.

Note that the ballot definition contains not only the "visible content" of the ballots, but also additional information, including for instance ballot validation rules (such as the maximum number of ballots for candidates). The ballot audit application can ignore this additional information, as it is expected to render the ballot as it is (as cast), without interpreting the validation and counting rules. The important point is that the displayed representation of the cast ballot should clearly communicate the recorder voter's choice (and nothing more).

## B.9 The Receipt

The signed acknowledgement included in the initial message (login response), along with the computed ballot fingerprint, can be now (transformed to the appropriate format) and offered to the voter for download or given directly to the auditors (who can then make sure that the voter's ballot is included in the tally).

See Section C.2 for the expected format of the receipt files.

Note that we assume here that this acknowledgement has been already checked, as described in Section B.6.

## B.10 Second Device Protocol: Example Transcript

This section presents an example transcript of the second device protocol carried out between the second device application and the election system backend.

### B.10.1 The content of the QR code

```
encC  = J8kdvzMZHPQBHbVoNx+JrstXlgbhOqDIikjbOGOL9HvWVdRPWuw/Xbvq/nKS/mqH/h2FJjCYbozkJiq7
encD  = /EhRgDjIA+scXsSyfSXvqPCHsvbf/UozQicLbNd4bjkps8aP4ZXdo3R+KuvYX/ZM8NeAJcGrZbeb3wm8fgnby1gQJGqJwMY+eN6qXN83b0i5pNae
vid   = voter7
nonce = e552502592f5bec54e4750c769ae9a3ec913c69a7cd828ce0226201476a2f833
```

### B.10.2 The election status response

The backend system provides an election status endpoint, which does not require any authentication and returns minimal information about the election (title and the used languages). This information is intended to be used on the login page of the second-device application.

```
{
```

```
  "title": {
    "default": "My Election Title",
    "value": {}
  },
  "languages": [
    "EN",
    "DE"
  ]
}
```


### B.10.3 The login request


```
{
  "voterId": "voter7",
  "ballotReference": "/EhRgDjIA+scXsSyfSXvqPCHsvbf/UozQicLbNd4bjkps8aP4ZXdo3R+KuvYX/ZM8NeAJcGrZbeb3wm8fgnby1gQJGqJwMY+eN
  "nonce": "e552502592f5bec54e4750c769ae9a3ec913c69a7cd828ce0226201476a2f833",
  "password": "711852",
  "challengeCommitment": "02438c535a05445cac1b15e017cc0755babaedd418d6bd581d51b87aae93a9fd14"
}
```


### B.10.4 The login response


```
{
  "token": "dm90ZXI3.aXhCaVJFRWtlWVpzRUVpNw==",
  "ballotVoterId": "voter7",
  "electionId": "0d6533e3-4d3f-4756-83c7-b03765460f62",
  "languages": [
    "EN",
    "DE"
  ],
  "title": {
    "default": "My Election Title",
    "value": {}
  },
  "contentAbove": {
    "value": {
      "default": "This is content above",
      "value": {}
    },
    "contentType": "TEXT"
  },
  "publicLabel": "A",
  "messages": {},
  "allowInvalid": true,
  "initialMessage": "{\"secondDeviceParametersJson\":\"{\\\"publicKey\\\":\\\"0390c059a207899b2dd76d5f5b7f40c73a02e620b6
}
```


where the field `initialMessage` contains a JSON string with the following object:


```
{
  "secondDeviceParametersJson": "{\"publicKey\":\"0390c059a207899b2dd76d5f5b7f40c73a02e620b67a5a23cc07134c3462b659aa\",\
```

```
  "comSeed": "6b08702a1930646f9fbb80c9f57ba75a02110939195d6b5f7790062ac86b5e82",
  "ballot": {
    "encryptedChoice": {
      "ciphertexts": [
        {
          "x": "0223b5b92fe26dc525e2aef344ec247f4c2cd35eac2a9748768e9487dd6056ab03",
          "y": "0368514b71bc2e243850328652ce99e54ae0176daec225489a35011037a1c35362"
        }
      ]
    },
    "zkp": [
      {
        "c": "947954522797530459513438643019753459753727999365552815563890082248808437660 8",
        "f": "552105630166835981446601515614868642452464030125998736951775236433233274021 1"
      }
    ],
    "publicLabel": "A",
    "reference": "aENXTx-4eutmE-L32MeZ-ehag",
    "signature": {
      "c": "950617761605033240999486766486157615879887377470864211701212274055317824848 38",
      "f": "610804248638566273474237987371037343151762369952805900445288972833238011370 66"
    }
  },
 "signatureHex": "44f547c20f60ce2a67dbd19c342530f836d4e1727573c38ca4844db73ae88b8495692cbd1d56db0724c56a9818f6a9e320fd4
  "factorX": [
    "02c6b7cead1a06f0563352de0592a80e9c22210551c07ce3b38837c789393c80d6"
  ],
  "factorY": [
    "03abf72793cad412da56b977b128de9a3097e52a3ceecfa0aadc2d233727c9b105"
  ],
  "factorA": [
    "035fda9e504e0c1b8275d9d920e729a52a86bbb6a5fc1c679ac51cbdbc56235780"
  ],
  "factorB": [
    "0346f00259f21c9288510a173d546247d849138c30f17bcde4322165bf5be34c8a"
  ]
}
```


where the field `secondDevicePublicParametersJson` contains a JSON string with the following object (where `ballots` is a list of elements of type `Core3Ballot`; see Section ??):


```
{
  "publicKey": "0390c059a207899b2dd76d5f5b7f40c73a02e620b67a5a23cc07134c3462b659aa",
 "verificationKey": "308201a2300d06092a864886f70d01010105000382018f003082018a0282018100db68673690d266f8ce7c0b718ca3be22
  "ballots": [
    {
      "type": "STANDARD_BALLOT",
      "id": "A",
      "title": {
        "default": "Ballot title",
        "value": {}
      },
      "lists": [
        {
```

```
    "id": "A1",
    "title": {
      "default": "First question!",
      "value": {}
    },
    "columnHeaders": [
      {
        "default": "",
        "value": {}
      }
    ],
    "candidates": [
      {
        "id": "A1-1",
        "columns": [
          {
            "value": {
              "default": "Yes",
              "value": {}
            },
            "contentType": "TEXT"
          }
        ],
        "maxVotes": 1,
        "minVotes": 0
      },
      {
        "id": "A1-2",
        "columns": [
          {
            "value": {
              "default": "No",
              "value": {}
            },
            "contentType": "TEXT"
          }
        ],
        "maxVotes": 1,
        "minVotes": 0
      }
    ],
    "maxVotesOnList": 1,
    "minVotesOnList": 1,
    "maxVotesForList": 0,
    "minVotesForList": 0,
    "voteCandidateXorList": false
  }
],
"showInvalidOption": true,
"showAbstainOption": false,
"maxVotes": 1,
"minVotes": 0,
"prohibitMoreVotes": false,
"prohibitLessVotes": false,
"calculateAvailableVotes": false,
"voterClientSettings": {
  "calculateAvailableVotes": false,
  "hideVoteCountForLists": false,
```

```
        "showInvalidOption": true,
        "showAbstainOption": false,
        "prohibitMoreVotes": false,
        "prohibitLessVotes": false
      }
    }
  ]
}
```


### B.10.5 The challenge request


```
{
  "challenge": "430304450494877475410297262357312327212929068391862527553207098479828124727 54",
 "challengeRandomCoin": "265754957419350644550705033009951014278994817004808313299694248829112365409 02"
}
```


### B.10.6 The challenge response:


```
{
  "z": [
   "440140691428869804775717914092066196580154800239546908099299395598769943680581768878098633279893671587503598960084 0
  ]
}
```


# C Additional Verification Tasks

## C.1 Second Device Public Parameters

When the election system is deployed and bootstrapped, it creates the so called *second-device public parameters*, that is parameters of the voting process relevant for the ballot receipt signing/verification and ballot audit (cast-as-intended verification with a second device). These parameters include:

- the public election key (used to encrypt ballots),
- the receipt verification key (used by the main voting device and by the second device to check signatures on the ballot receipts issued by the voting system to the voter),
- the content of the ballots used in the election.

The e-voting system offers these public parameters, along with their fingerprint (the SHA- 512 hash of the JSON representation of the second-device public parameters), for download via the Election Admin Panel. If a second device application is deployed

(for the cast-as-intended ballot audit), such an application should be pre-configured with this fingerprint.

A tool for universal verifiability should offer the possibility of checking that a given file with the public parameters fingerprint is consistent with the content of the bulletin boards. This process is described below.

**The format.** A file downloaded via the EAP provides the aforementioned public parameters and the corresponding fingerprint in the following JSON format:

```
{
    "publicParametersJson": PUBLIC_PARAMETERS_JSON,
    "fingerprint": FINGERPRINT
}
```

where `PUBLIC_PARAMETERS_JSON` is a string with the JSON representation of the public parameters, as described below, and `FINGERPRINT` contains the SHA256 hash of the `PUBLIC_PARAMETERS_JSON` as a hexadecimal string.

The public parameters are of the form

```
{
    "publicKey": PUBLIC-ELECTION-KEY,
    "verificationKey": RECEIPT-VERIFICATION-KEY,
    "ballots": BALLOTS
}
```

where `PUBLIC-ELECTION-KEY` is the public election key (X.509 encoded and represented as a hexadecimal string),
`RECEIPT-VERIFICATION-KEY` is a (hexadecimal representation) of an RSA public key for checking signatures on ballot receipts, and `BALLOTS` is a list of ballots offered in the election.

For instance, the content of a downloaded file with public parameters can have the following format:


```
{
   "publicParametersJson":"{\"publicKey\":\"026ad8b0f428265c0e491856093a7b9b2ad9411da053bb844fe1f62b2af4a7cc93\"....
   "fingerprint":"979c947e10bf15d69d38595c4ae7f91b6214578264be099b3abbd5339117fedbd77c1596ec9d79769637d8701b528f....
}
```


**Verification.** The verification procedure should check that:

1. The fingerprint `FINGERPRINT` is in fact the SHA-256 hash of `PUBLIC_PARAMETERS_JSON`

2. The data in PUBLIC-ELECTION-KEY, RECEIPT-VERIFICATION-KEY, and BALLOTS is consistent with the content of the bulletin boards. That is, the verification procedure should make sure that:

    - The value PUBLIC-ELECTION-KEY is equal to the public key published on board keygen-electionKey-Polyas (Section E.6.2).
    - The value RECEIPT-VERIFICATION-KEY is equal to the verification key published on board BBox-ballotbox-key-CP (Section E.3.2).
    - The value BALLOTS is the list of ballots, as published on board registry (Section E.1.2), in the field ballotStructures.

## C.2 Receipts

The voters have the option to save *ballot cast confirmations* (receipts), containing fingerprints of their encrypted ballots, signed by the election system. Such a receipt does not reveal the plaintext voter's choice (how the voter voted) and can therefore be safely handed out to third parties for audit.

A voter can download a ballot cast confirmation using the main voting application (right after the ballot cast step), as well as using the second device application, if the cast-as-intended mechanism with the second device is employed in the election.

The process can be organized in such a way that a voter can hand over their receipt to auditors (who carry out the universal verifiability procedure) in order to make sure that their ballot is included in the final tally (that is in the verification package which is used to confirm correctness of the final tally). To support this process, a verification tool needs to be able to parse receipt files and make the required consistency checks, as detailed below.

**Format of the receipt file.** Receipts (ballot cast confirmations) are downloaded by the voters as simple PDF files containing a signed ballot fingerprint in the following format:

```
--BEGIN FINGERPRINT--
5a65b1192c8ddbc5a19ac6172c26b69561abdc5d5fdaffc8e47059980ab2cef1
--END FINGERPRINT--
--BEGIN SIGNATURE--
08ce02e615089992965080a2004e564908891ee4dafd0d2f7ef5e2590b9da988
0bcd7f621d84b01e75e88dfba6bcede7c479258b46287ffbcefae601ef8ea65d
3efac01842757921241bbb9e0a9e97e82aea67eee1857cc91edebf6736402c45
a284853c844aa4bd9f1404cb8a7faf9208016b1b6165a22bb36b3793f3467f72
3af83920d6383b37bcfb0bdf06916d459371dbd85322f1c41c50269d0c365440
c7f162f9cc3b507c35c92d4b22162a5683062f975fd6a65961aebc53af185af2
```

```
44cddce27236e6df513631258e116bb3a5947e1afe8354683a89a6354d8b4e15
1ca373fade2a86598ea11124af18b4485f9a215cc5053773b393e8b938429c59
--END SIGNATURE--
```

The first part contains the fingerprint of the encrypted ballot and the second one contains an RSA signature of this fingerprint, created by the election system. The verification application should extract this block from the PDF and apply the checks described below.

The above data is sufficient for the verification process. The recipt may however include additional information for convenience of the receipt handling, such as the *ID of the election* and the *anonymous voter ID*. Importantly however, **the receipt should not include the public voter identifier** (*vid*).

*Note.* The verification key for checking this signature is included in the second device public parameters, as defined above.

**Verification.** With a fingerprint and a signature extracted from the receipt file, the verification application should carry out the following steps:

1. Check that the included signature is a correct signature on the included fingerprint, using the *receipt verification key* published on board BBox-ballotbox-key-CP (Section E.3.2).

   If this check fails, report a warning (the provided file does not contain a correct signature of the election system). As, in this case, it may be impossible to resolve the issue (determine if the voter intentionally provided a fake receipt or if it was the voting application/second device application that generated invalid file), the election council may need to define a process to handle such cases.

2. If the above check has succeeded (the signature is correct), check that a ballot entry with the provided fingerprint is included in the ballot-box (Section E.1.3). In order to compute a fingerprint of an entry of the ballot box (of type BallotEpEntry (Section F.4) apply the SHA256 hash function to the following data (using the conventions of Section A.1.2 for transforming the input data into the byte array to be hashed):

   - the public label as a byte array (using the UTF-8 encoding), prepended with its length (as a 4-byte integer),
   - the public credentials as a byte array, prepended with its length,
   - the voter ID as a byte array (using the UTF-8 encoding) prepended with its length,

- the number of elements in the list of ciphertexts `ballot.encryptedChoice.ciphertexts` (as 4-byte integer) (see also Ballot (Section **??**))
- the $x$ and then the $y$ components of each element of this list, represented as byte arrays prepended with their lengths,
- the number of elements in the list `ballot.proofOfKnowledgeOfEncryptionCoins`,
- the `c` and then the `f` component of each element of this list, represented as byte arrays prepended with their lengths,
- `proofOfKnowledgeOfPrivateCredential.c` and `proofOfKnowledgeOfPrivateCredential.` represented as byte arrays prepended with its length.

and take the hexadecimal representation of this hash.

For example, for the ballot entry


```
{
  "publicLabel" : "A",
  "voterID" : "03b36cfc60f1fa86a5826bb5377fe6ec123045df33652516101ae803cca57b4276",
  "ballot" : {
    "encryptedChoice" : {
      "ciphertexts" : [ {
        "x" : "0201df95626718539000d65a8049f2418328df95a61107357b96db2f5dc304b38b",
        "y" : "03692902cdfc6febebcd1d175859a6ea84018fe47c345f5e44fffddf97a492112f"
      } ]
    },
    "proofOfKnowledgeOfEncryptionCoins" : [ {
    "c" : "10188446347544979212343588959157565121623778797627805375625466440061560984940 2",
      "f" : "81174399198084050819276673106811016645306940417447240994496938329622216775741"
    } ],
    "proofOfKnowledgeOfPrivateCredential" : {
     "c" : "7444867864096567261070659423887179991345588536464902181904800065159483964442 6",
      "f" : "33276182696803028530168279419966710636866170477238993593368673329284148491287"
    }
  },
  "publicCredential" : "03b36cfc60f1fa86a5826bb5377fe6ec123045df33652516101ae803cca57b4276"
}
```


the digested bytes are


```
0000000141000000210 3b36cfc60f1fa86a5826bb5377fe6ec123045df336525
16101ae803cca57b427600000042303362333663666336306631666138366135
383236626235333737666536656331323330343564663333363532353136313 0
3161653830336363613537623432373600000001000000210201df9562671853
9000d65a8049f2418328df95a61107357b96db2f5dc304b38b00000021036929
02cdfc6febebcd1d175859a6ea84018fe47c345f5e44fffddf97a492112f0000
00010000002100e1409011d385f929f77566ed7bbf9e60677ed5946f7872931f
0ae564c4b1d63a0000002100b37714efd6c1c9cedaa0b3eb8c8d53c9aaec3e5e
1785073214b6623ae975183d0000002100a49875774197101742008 3c6c2ebd0
e6718c03d4e168dbd3f7859ed13894590a000000204991a6e74dc5ed7012c9aa
1a08d9e1b75a49e839abedff068b94419e5fe9f417
```

and the fingerprint is

```
534a6f16ae3f3e77e971d16bc4893c680824b5f16a882dfe299c629e33870dc3
```

# D Format of the Verification Data

In this section we describe the data format of the data packet provided by the Election Provider to the Election Council at the end of the election. This data packet contains the content of all the bulletin boards used during the election process. This includes, in particular, the content of the ballot box, the registry board, the final (raw) result (decrypted ballots), and the intermediate data with zero-knowledge proofs.

The data packet is in the ZIP format. This file contains (among others), for each bulletin board with name *board-name*, a file named *board-name*`.json`. Such a file contains list of (JSON) records with the content explained below.

**Note.** Polyas routinely uses *secure bulletin boards* to store data (ballot box, the output of the intermediate steps, and the final result). Secure bulletin boards use digital signatures and hash chains in order to enhance integrity of the process (only new entries published by an authorised component are supposed to be appended to a bulletin board; no entries are supposed to be deleted or modified). Checking integrity of bulletin boards, by auditing the hash chains and digital signatures, is *not* subject of the verification task as described in this document (although the verification task can be extended in the future to include the verification of integrity of bulletin boards). Therefore, in the following description, we do not explain those elements of the data files which have to do with bulletin board integrity.

Each record in the file is for the form

```
{
    ...
    "c" : content
    ...
}
```

where the field labeled with `"c"` points to the content of the given record in JSON format (the remaining fields contain data related to bulletin board integrity and are, as mentioned, not discussed here). The type of the content depends on the bulletin board.

# E Bulletin Boards

In this section we list all the bulletin boards used in the described instance of Polyas Core 3.0.

We distinguish *external* boards as a separate category, where such information is published as ballots (produced by voting clients) and registry record (uploaded by the Election Council). The content of the remaining boards is computed during the election process from the content of another boards.

A bulletin board can be blocked by so called *triggers*, which means that the system waits for some conditions before any data is published on the board. One type of triggers is related to election phase changes (some computations should only happen after some election phase is reached). Another type of a trigger is so-called guard trigger. In this case some computations wait till all the data from some selected boards is fully verified. For instance, decryption may have to be postponed until the complete mixing process has been verified.

**Authorities**

The operations are performed by the following *authorities*, where CP is the *election controller* owning the ballot box and other external boards:

- CP
- Polyas

**Sub-components**

The system consists of the following *sub-components* (logical groups of boards):

- ballot
  Ballot pre-processing which flags and filters out incorrect ballots (for instance, ballots with invalid zero-knowledge proofs) and revoked ballots (if the election supports revocations) from the ballot box.
- BBox
  Boards belonging to the election controller (CP), related to the ballot box operations.
- decryption
  Ballot decryption
- final-guard
  Verification module (checking correctness of operations)

- keygen
  ElGamal key pair generation
- mixing
  Verifiable shuffle with encrypted ballots carried out by authorities Polyas
- pre-decryption-guard
  Verification module (checking correctness of operations)

## E.1 External boards

The input to the system is provides via the external boards listed below. This input is assumed to be provided by external control components. In particular, an appropriate component publishes ballots cast by voters on the board representing the ballot box.

Below, we list the external triggers and then provide details on the remaining external boards.

### E.1.1 Triggers

- **init-trigger**
  Actors waiting for this trigger: keygen-secretKey-Polyas
- **registration-trigger**
- **tally**
  Actors waiting for this trigger: decryption-decrypt-Polyas
- **voting-trigger**

### E.1.2 External board registry

**Content:** An entry of type `EpRegistry<ECElement>`

**Entry description:** Registry entry containing, in particular, the ballot description and the list of registered voters

**Example content**


```
{
  "packetSize" : 400,
  "electionId" : "election-id",
  "electionTitle" : "Test registry (1791555230372)",
  "ballotStructures" : [
    {
      "id" : "0",
```

```
    "encoding" : {
      "type" : "Standard"
    },
    "title" : {
      "default" : "ballot 0",
      "value" : { }
    },
    "lists" : [
      {
        "id" : "0-0",
        "title" : {
          "default" : "List 0",
          "value" : { }
        },
        "columnHeaders" : [
          {
            "default" : "List 0",
            "value" : { }
          }
        ],
        "candidates" : [
          {
            "id" : "0-0-0",
            "columns" : [ ],
            "maxVotes" : 1,
            "minVotes" : 0
          },
          {
            "id" : "0-0-1",
            "columns" : [ ],
            "maxVotes" : 1,
            "minVotes" : 0
          }
        ],
        "maxVotesOnList" : 999,
        "minVotesOnList" : 0,
        "maxVotesForList" : 999,
        "minVotesForList" : 0,
        "voteCandidateXorList" : false
      }
    ],
    "showInvalidOption" : false,
    "showAbstainOption" : false,
    "maxVotes" : 999,
    "minVotes" : 0,
    "prohibitMoreVotes" : false,
    "prohibitLessVotes" : false,
    "calculateAvailableVotes" : false,
    "type" : "STANDARD_BALLOT"
  }
],
"voters" : [
  {
    "id" : "KwYlYn-352RPQ-LIK3Ge-LwtF",
    "publicLabel" : "0",
    "cred" : "03d2bfccbd8434dc84d0f1604c0d7e0c1b59fb44022b8a4daca10f457105d015b6"
  },
  {
```

```
    "id" : "higaEr-lMlII2-JQJ2kg-Pt9v",
    "publicLabel" : "0",
    "cred" : "02f63d314db40c1ae168c3967b252a6b678b91a5c6da98774bc2205791ab7c8c57"
  },
  {
    "id" : "kHZFG0-0C9xrF-ipKt1U-FXnk",
    "publicLabel" : "0",
    "cred" : "020d7a2145b31b64eea33c26ef1e4117a3eb7b5c9404d727230a71b10f94a9a26c"
  },
  {
    "id" : "J+Krgg-IAwv6a-KupNk+-ogvX",
    "publicLabel" : "0",
    "cred" : "02d58bc46fe8220c3d966a33ccb465047d7fa5b8d1d526ff4b1381d3961a12d835"
  },
  {
    "id" : "kih/KA-zH5FkF-We37T4-H+TT",
    "publicLabel" : "0",
    "cred" : "034608a27f0b8dbae2db60cc0b1abb9150c8016767b3342efc2aeedf34f3b62db2"
  },
  {
    "id" : "PG1u0h-5hmXUq-Jq0wnH-b/tN",
    "publicLabel" : "0",
    "cred" : "020b17d5bff67af23f2086c2898475127f93e5c89155ff3af0a1e677aa9bbf4bd3"
  },
  {
    "id" : "AF1jHs-/40kq3-Ng5c3u-dDqU",
    "publicLabel" : "0",
    "cred" : "03e02a7c3827ced3e6de231e2149bf283662e6c6e14a6c15b5bfacf20475d4b9d6"
  },
  {
    "id" : "f9jlL5-oDkaiR-kkB2A4-fdss",
    "publicLabel" : "0",
    "cred" : "0391913f5fcda04efec6bb634962f3bf4a9c5948ffdb53fda67bf60ae68ea214b5"
  }
],
"voterIdentifiers" : [
  {
    "publicId" : "voter-0",
    "publicLabel" : "0"
  },
  {
    "publicId" : "voter-1",
    "publicLabel" : "0"
  },
  {
    "publicId" : "voter-2",
    "publicLabel" : "0"
  },
  {
    "publicId" : "voter-3",
    "publicLabel" : "0"
  },
  {
    "publicId" : "voter-4",
    "publicLabel" : "0"
  },
  {
    "publicId" : "voter-5",
```

```
      "publicLabel" : "0"
    },
    {
      "publicId" : "voter-6",
      "publicLabel" : "0"
    },
    {
      "publicId" : "voter-7",
      "publicLabel" : "0"
    }
  ],
  "ballotSigningKey" : "308201a2300d06092a864886f70d01010105000382018f003082018a0282018100c804b11e124685a1c3ced4d007ae0f4
}
```


### E.1.3 External board ballot-box

**Content:** Sequence of entries of type `BallotEpPackage<ECElement>`

**Entry description:** Ballot box entry containing a list of individual (encrypted) ballots

**Example content**


```
- {
    "ballots" : [
      {
        "ballot" : {
          "encryptedChoice" : {
            "ciphertexts" : [
              {
                "x" : "0308626f5d03c0cfaf5cb14dd692bad031f8c2ec6b91a5a99500ebee6156d2de22",
                "y" : "02843863f6393dc8aa535d12e2e75510b53ce4ac4e7ab556a44b34f8fdfae06c4a"
              }
            ]
          },
          "zkp" : [
            {
              "c" : "37418243854815207385774745425275720764099822916951461347635221987576752630 44",
              "f" : "55202894728659786632467892210899249241504602904394150502538148495845971390065"
            }
          ],
          "publicLabel" : "0",
          "reference" : "KwYlYn-352RPQ-LIK3Ge-LwtF",
          "signature" : {
            "c" : "380842197497724522136369807175658348144135727955857785704177433844086263334 77",
            "f" : "32351089618838633851605660370289604472865423071475620696127442292984601231167"
          }
        }
      },
      {
        "ballot" : {
          "encryptedChoice" : {
            "ciphertexts" : [
              {
                "x" : "034c6d77e7cc0af6703987ffd23a80bde6334e8e9956b1ee11965b386769a3d7b2",
```

```
              "y" : "023cc717456a0e8b0626fd9f3a0467559a7b1f04b90fc9cd92b9df4d94994aa793"
            }
          ]
        },
        "zkp" : [
          {
            "c" : "110444814682998166709261323550283459264252877215153365130579982318702740727185",
            "f" : "333089335751526474741470742244208281082468867841763037707999842852495097137O4"
          }
        ],
        "publicLabel" : "0",
        "reference" : "kHZFG0-0C9xrF-ipKt1U-FXnk",
        "signature" : {
          "c" : "36131450668988877567204590571604797624242061510196046910058235445155807620450",
          "f" : "449513774724846431568952116316429835926849854215781279150462633802005659471O8"
        }
      }
    }
  ]
}
- {
  "ballots" : [
    {
      "ballot" : {
        "encryptedChoice" : {
          "ciphertexts" : [
            {
              "x" : "0256a6eafb78bc376a3a68c100970982454c94b628262ba2ef524b3bf995c13c44",
              "y" : "02b88e19f371b22780c484bb7f816c1e1bf17c8619cb646878f61367caeb00a76a"
            }
          ]
        },
        "zkp" : [
          {
            "c" : "413343608121350572929012740145689080325385595491256697167114028459203655527O5",
            "f" : "78850137656006163120819612932315437037611466463606656200790884507674631114462"
          }
        ],
        "publicLabel" : "0",
        "reference" : "kih/KA-zH5FkF-We37T4-H+TT",
        "signature" : {
          "c" : "525220023648192910641315591440174655694968561913106464273034255769528171019l4",
          "f" : "103205972590605107157658876732216005219661676199007448372270133614488067703455"
        }
      }
    },
    {
      "ballot" : {
        "encryptedChoice" : {
          "ciphertexts" : [
            {
              "x" : "036076551cf8806f7e0e91fa196a6cffde5061770db700a5f2906ac1bcc0554451",
              "y" : "026e7671a19a8fbb6723d8df0ef32d0aa84a0e443987b3dcd00958886fada95d21"
            }
          ]
        },
        "zkp" : [
          {
```

```
            "c" : "563618391839225906458930832946185976922250505050010052939784759004671072155 67",
            "f" : "105808930770484876357192658912577836739148477916001292188785508843905491174 55"
          }
        ],
        "publicLabel" : "0",
        "reference" : "AF1jHs-/40kq3-Ng5c3u-dDqU",
        "signature" : {
          "c" : "286090502456617819661271673066263890533888922788148952249517443978532763225 9",
          "f" : "452658099679289255652929203838119269627109808493118210928000385939133998756 78"
        }
      }
    }
  ]
}
```


### E.1.4 External board revocations

**Content:** Sequence of entries of type `RevocationToken`

**Entry description:** A revocation token specifying voters whose ballots are to be revoked

**Example content**


```
- {
    "electionId" : "election-id",
    "voterIds" : [
      "higaEr-lMlII2-JQJ2kg-Pt9v",
      "kHZFG0-0C9xrF-ipKt1U-FXnk"
    ]
  }
- {
    "electionId" : "election-id",
    "voterIds" : [
      "f9jlL5-oDkaiR-kkB2A4-fdss"
    ]
  }
```


## E.2 Sub-component ballot

Ballot pre-processing which flags and filters out incorrect ballots (for instance, ballots with invalid zero-knowledge proofs) and revoked ballots (if the election supports revocations) from the ballot box.

### E.2.1 Board ballot-flagged

**Sub-component:** ballot

**Board name:** ballot-flagged

**Content:** Entries of type `Packet<Annotated<BallotEpEntry<ECElement>>>`

*Description:* Ballots from the ballot box, along with an explicit annotation about their classification as correct, incorrect, or revoked. Published in packets.

**Used by:**

- ballot-filtered-out
- mixing-input-packets

### Public input

- ballot-box — entries of type `BallotEpPackage<ECElement>`
- revocations — entries of type `RevocationToken`
- registry — an entry of type `EpRegistry<ECElement>`
- keygen-electionKey-Polyas — an entry of type `PublicKeyWithZKP<ECElement>`

### Example content


```
- {
    "values" : [
      {
        "ballot" : {
          "ballot" : {
            "encryptedChoice" : {
              "ciphertexts" : [
                {
                  "x" : "0308626f5d03c0cfaf5cb14dd692bad031f8c2ec6b91a5a99500ebee6156d2de22",
                  "y" : "02843863f6393dc8aa535d12e2e75510b53ce4ac4e7ab556a44b34f8fdfae06c4a"
                }
              ]
            },
            "zkp" : [
              {
                "c" : "37418243854815207385774745425275720764099822916951461347635221987576752630 44",
                "f" : "55202894728659786632467892210899249241504602904394150502538148495845971390065"
              }
            ],
            "publicLabel" : "0",
            "reference" : "KwYlYn-352RPQ-LIK3Ge-LwtF",
            "signature" : {
              "c" : "380842197497724522136369807175658348144135727955857785704177433844086263334 77",
              "f" : "32351089618838633851605660370289604472865423071475620696127442292984601231167"
            }
          }
        },
        "status" : "OK",
        "annotation" : ""
      },
```

```
{
  "ballot" : {
    "ballot" : {
      "encryptedChoice" : {
        "ciphertexts" : [
          {
            "x" : "034c6d77e7cc0af6703987ffd23a80bde6334e8e9956b1ee11965b386769a3d7b2",
            "y" : "023cc717456a0e8b0626fd9f3a0467559a7b1f04b90fc9cd92b9df4d94994aa793"
          }
        ]
      },
      "zkp" : [
        {
         "c" : "1104448146829981667092613235502834592642528772151533651305799823187027407271 85",
          "f" : "333089335751526474741470742244208281082468867841763037707999842852495097137 04"
        }
      ],
      "publicLabel" : "0",
      "reference" : "kHZFG0-0C9xrF-ipKt1U-FXnk",
      "signature" : {
        "c" : "36131450668988877567204590571604797624242061510196046910058235445155807620450",
        "f" : "44951377472484643156895211631642983592684985421578127915046263380200565947168"
      }
    }
  },
  "status" : "REVOKED",
  "annotation" : "The voter kHZFG0-0C9xrF-ipKt1U-FXnk has been revoked."
},
{
  "ballot" : {
    "ballot" : {
      "encryptedChoice" : {
        "ciphertexts" : [
          {
            "x" : "0256a6eafb78bc376a3a68c100970982454c94b628262ba2ef524b3bf995c13c44",
            "y" : "02b88e19f371b22780c484bb7f816c1e1bf17c8619cb646878f61367caeb00a76a"
          }
        ]
      },
      "zkp" : [
        {
          "c" : "41334360812135057292901274014568908032538559549125669716711402845920365552735",
          "f" : "78850137656006163120819612932315437037611466463606656200790884507674631114462"
        }
      ],
      "publicLabel" : "0",
      "reference" : "kih/KA-zH5FkF-We37T4-H+TT",
      "signature" : {
        "c" : "52522002364819291064131559144017465569496856191310646427303425576952817101914",
        "f" : "10320597259060510715765887673221600521966167619900744837227013361448806770345 5"
      }
    }
  },
  "status" : "OK",
  "annotation" : ""
},
{
  "ballot" : {
```

```
        "ballot" : {
          "encryptedChoice" : {
            "ciphertexts" : [
              {
                "x" : "036076551cf8806f7e0e91fa196a6cffde5061770db700a5f2906ac1bcc0554451",
                "y" : "026e7671a19a8fbb6723d8df0ef32d0aa84a0e443987b3dcd00958886fada95d21"
              }
            ]
          },
          "zkp" : [
            {
              "c" : "563618391839225906458930832946185976922250505050010052939784759004671072155 67",
              "f" : "105808930770484876357192658912577836739148477916001292188785508843905491174 55"
            }
          ],
          "publicLabel" : "0",
          "reference" : "AF1jHs-/40kq3-Ng5c3u-dDqU",
          "signature" : {
            "c" : "286090502456617819661271673066263890533888922788148952249517443978532763225 9",
            "f" : "452658099679289255652929203838119269627109808493118210928000385939133998756 78"
          }
        }
      },
      "status" : "OK",
      "annotation" : ""
    }
  ]
}
```


### E.2.2 Board ballot-filtered-out

**Sub-component:** ballot

**Board name:** ballot-filtered-out

**Content:** Entries of type `String`

*Description:* All the ballots from the ballot box which have been filtered out (flagged as incorrect or revoked)

**Used by:** *none*

**Public input**

- ballot-flagged — entries of type `Packet<Annotated<BallotEpEntry<ECElement>>>`

**Example content**

```
- "The voter kHZFG0-0C9xrF-ipKt1U-FXnk has been revoked."
```

## E.3 Sub-component BBox

Boards belonging to the election controller (CP), related to the ballot box operations.

### E.3.1 Secret BBox-ballotbox-key-secret-CP

*Computed by authority CP*

**Sub-component:** BBox

**Name:** BBox-ballotbox-key-secret-CP

**Content:** An entry of type `SigningKeyPair`

*Description:* Secret (signing) RSA key used by the election system to sign ballot receipts

**Used by:**

- BBox-ballotbox-key-CP

### E.3.2 Board BBox-ballotbox-key-CP

*Computed by authority CP*

**Sub-component:** BBox

**Board name:** BBox-ballotbox-key-CP

**Content:** An entry of type `VerificationKey`

*Description:* Public (verification) RSA key for checking signatures of the election system on the ballot receipts

**Used by:** *none*

**Secret input**

- BBox-ballotbox-key-secret-CP — an entry of type `SigningKeyPair`

**Example content**


```
"308201a2300d06092a864886f70d0101010500038201f003082018a0282018100c804b11e124685a1c3ced4d007ae0f4cfe7b6f399aa7daf33d8c
```

## E.4 Sub-component decryption

Ballot decryption

### E.4.1 Board decryption-decrypt-Polyas

*Computed by authority Polyas*

**Sub-component:** decryption

**Board name:** decryption-decrypt-Polyas

**Content:** Entries of type `MessageWithProofPacket<ECElement>`

*Description:* Result of the decryption of the input multi-ciphertexts (from the input ciphertext packets) along with zero-knowledge proofs of correct decryption

*Generic description of the entry type:* A packet of decrypted messages with zero-knowledge proofs of correct decryption

**Used by:** *none*

**Waits for (is blocked by)**

- pre-decryption-guard-Polyas
- tally (a trigger)

**Secret input**

- keygen-secretKey-Polyas — an entry of type `BigInteger`

**Public input**

- mixing-mix-Polyas — entries of type `MixPacket<ECElement>`

**Additional input for verification**

- keygen-electionKey-Polyas — an entry of type `PublicKeyWithZKP<ECElement>`

**Example content**

```
- {
    "publicLabel" : "0",
    "messagesWithZKP" : [
```

```
    {
      "message" : "00000101",
      "proof" : [
        {
          "decryptionShare" : "03f5e9c033b352bdcd64e756f6efc235b96a90a1d0870195eab47cb0513022e5f5",
          "eqlogZKP" : {
            "c" : "22223961005505880578966682873221828823392960169376722693583779559489701107390",
            "f" : "59845973738669108623070199844884448796693000402869328900609629702132566000012"
          }
        }
      ]
    },
    {
      "message" : "00000101",
      "proof" : [
        {
          "decryptionShare" : "02b271cf5277d92b02a713cb3b1cb0c1fe5b7c39f830a8851d954f4bdd263ca6ad",
          "eqlogZKP" : {
            "c" : "91731140672073097789081710455049943836093062665203659319831837034312568253263",
            "f" : "10863926166883806168027381587850606437688476632016232927337504706756183987 3703"
          }
        }
      ]
    },
    {
      "message" : "00000101",
      "proof" : [
        {
          "decryptionShare" : "02fa1cc3c78ff40d399398abe83e620082a67e52cdd7e0ffb44ec42840c295e3e3",
          "eqlogZKP" : {
            "c" : "55629514272380049764326609167285925566999847481932765358375075244699859798123",
            "f" : "84881212622502938089285127264731601953624090545008792571018375382143894836 20"
          }
        }
      ]
    }
  ]
}
```


## E.5 Sub-component final-guard

Verification module (checking correctness of operations)

### E.5.1 Verification Component final-guard-Polyas

*Computed by authority Polyas*

**Sub-component:** final-guard

**Board name:** final-guard-Polyas

**Blocked boards (components which wait for this verifier)**: *none*

**Verified components:**

- decryption-decrypt-Polyas
- tally

## E.6 Sub-component keygen

ElGamal key pair generation

### E.6.1 Secret keygen-secretKey-Polyas

*Computed by authority Polyas*

**Sub-component:** keygen

**Name:** keygen-secretKey-Polyas

**Content:** An entry of type `BigInteger`

*Description:* The secret (decryption) key corresponding to the desc key

**Used by:**

- keygen-electionKey-Polyas
- decryption-decrypt-Polyas

**Waits for (is blocked by)**

- init-trigger (a trigger)

### E.6.2 Board keygen-electionKey-Polyas

*Computed by authority Polyas*

**Sub-component:** keygen

**Board name:** keygen-electionKey-Polyas

**Content:** An entry of type `PublicKeyWithZKP<ECElement>`

*Description:* The public desc key (used to encrypt ballots)

*Generic description of the entry type:* Public election key along with a zero-knowledge

proof of knowledge of the corresponding private key

**Used by:**

- ballot-flagged
- mixing-mix-Polyas
- decryption-decrypt-Polyas

**Secret input**

- keygen-secretKey-Polyas — an entry of type `BigInteger`

**Example content**


```
{
  "publicKey" : "03404362d167845408182706a5eea0c727e3a5809bc17ac6d409401d0373240611",
  "zkp" : {
    "c" : "78989242585042531547473731444011053162665497935421890550059443093828344560360",
    "f" : "52498934569930344052186560707169369348454448363386096094129519065115086371949"
  }
}
```


## E.7 Sub-component mixing

Verifiable shuffle with encrypted ballots carried out by authorities Polyas

### E.7.1 Board mixing-input-packets

**Sub-component:** mixing

**Board name:** mixing-input-packets

**Content:** Entries of type `MixPacket<ECElement>`

*Description:* Sequence of mix-packets containing ciphertexts to be shuffled, with the maximal packet size (that is the maximal number with ciphertexts in the packet) 400, obtained by grouping the ciphertexts (encrypted choices) from the input ballot entries by their public label and shared auxiliary data (which are recorded on the level of mix-packets). These mix-packets contain no zero-knowledge proofs.

*Generic description of the entry type:* Mix packet containing a list of ElGamal multi-ciphertexts plus, optionally, a zero-knowledge proof of correct shuffle

**Used by:**

- mixing-mix-Polyas

**Public input**

- ballot-flagged — entries of type `Packet<Annotated<BallotEpEntry<ECElement>>>`
- registry — an entry of type `EpRegistry<ECElement>`

**Example content**


```
- {
    "publicLabel" : "0",
    "ciphertexts" : [
      {
        "ciphertexts" : [
          {
            "x" : "0308626f5d03c0cfaf5cb14dd692bad031f8c2ec6b91a5a99500ebee6156d2de22",
            "y" : "02843863f6393dc8aa535d12e2e75510b53ce4ac4e7ab556a44b34f8fdfae06c4a"
          }
        ]
      },
      {
        "ciphertexts" : [
          {
            "x" : "0256a6eafb78bc376a3a68c100970982454c94b628262ba2ef524b3bf995c13c44",
            "y" : "02b88e19f371b22780c484bb7f816c1e1bf17c8619cb646878f61367caeb00a76a"
          }
        ]
      },
      {
        "ciphertexts" : [
          {
            "x" : "036076551cf8806f7e0e91fa196a6cffde5061770db700a5f2906ac1bcc0554451",
            "y" : "026e7671a19a8fbb6723d8df0ef32d0aa84a0e443987b3dcd00958886fada95d21"
          }
        ]
      }
    ]
  }
```


### E.7.2 Board mixing-mix-Polyas

*Computed by authority Polyas*

**Sub-component:** mixing

**Board name:** mixing-mix-Polyas

**Content:** Entries of type `MixPacket<ECElement>`

*Description:* Sequence of mix packets obtained by shuffling each of the input mix packets and producing the appropriate zero-knowledge proof with correct shuffle.

*Generic description of the entry type:* Mix packet containing a list of ElGamal multi-ciphertexts plus, optionally, a zero-knowledge proof of correct shuffle

**Used by:**

- decryption-decrypt-Polyas

### Public input

- mixing-input-packets — entries of type `MixPacket<ECElement>`
- keygen-electionKey-Polyas — an entry of type `PublicKeyWithZKP<ECElement>`

### Example content


```
- {
    "publicLabel" : "0",
    "ciphertexts" : [
      {
        "ciphertexts" : [
          {
            "x" : "03309d7059e80a71c6f96abab24f4b71b6e81939c8f66191b91d4bd411fdd61749",
            "y" : "02c841d01d727be09661a66b195e7d35155187e8a97e7df224d7b7b1812912193a"
          }
        ]
      },
      {
        "ciphertexts" : [
          {
            "x" : "03d7fb7bc4eae8f2a08ef28e07b370be32ad7f28bf7e7f7a206319db8af147ea54",
            "y" : "02cdc126c56744118e84e9e7540779d93ae24212a6cf06ed63c64bd0cdfa641a66"
          }
        ]
      },
      {
        "ciphertexts" : [
          {
            "x" : "038adb4c1ad736776984a44de9bc4ec8510c3f1fc1a0c2902dc0fe6367fe788cd1",
            "y" : "039d584a1cfebad37673842e6ee85f81cd269e27d3757cac108470d86765b9a79d"
          }
        ]
      }
    ],
    "proof" : {
      "t" : {
        "t1" : "03f0d0212d43bf3f4a2285b3c9c061858b7abebf8167912cde223ab25c4ecb2b39",
        "t2" : "03cf9e49c5b4fad9dca5f4b4745cb25c58b058dddf23158f28e2b40443a20ac8db",
        "t3" : "0358dd7ed5754899e440ee14c260dd8d63fd7b068b0ddd3c4f004886e9fa23c4db",
        "t4y" : [
          "02fb6c0e8b570e0ab745d6ff986f7081d297b6d1caeef7726ed71654744a4b2a71"
        ],
        "t4x" : [
          "032d601312461ac9e740cdd2092721f594fa0067b43b1d3cefd2e4069dab3c3d4d"
        ],
```

```
      "tHat" : [
        "03d92cdca943d33d153c1d394bb7b1fb57c5722cc76bdfead71ead30981ffe7083",
        "0381725bc54079d89e7fb68d8badc53b900e3e0a31fd432230d52763669c39c7e2",
        "020e5228e904d19e60665f0fb7cc004e432edc0003d709bc8c402dca2d84c662bc"
      ]
    },
    "s" : {
      "s1" : "300890593076580938144616428798387921883429822226777333559091404403822585552 24",
      "s2" : "240638298792178684857079207590649798661785567104191960278496732646693147997 27",
      "s3" : "564851694536664344021810326404172354008715184579416227597761144399242173461 46",
      "s4" : [
        "648231731590574951622812378022399328692485998876020525082632587451222342895 56"
      ],
      "sHat" : [
        "458881745689361542481204337514960053032258957328448777230188526751121342243 40",
        "829924114066402384248536097947309379186580873621449922670101595815658617933 91",
        "100924982292670935827141448674756610049664653538642042753472310955140391639 175"
      ],
      "sPrime" : [
        "158285335267905288844386056327458884173525227929717188595294345156395398470 89",
        "661268228037202556127611800233567886084240999092741529617582804982227576284 36",
        "144323836209701694457156183448544432310476141478776113161260494484286330015 56"
      ]
    },
    "c" : [
      "02bc5f03bada6f34ea2a53c5482f179025771aadc08037c93fcfe36fbb119f3697",
      "0297cd50bde09f2413e9be098c68e532c63cec929f0f61baa4b5129d46dbcca94d",
      "02eb886feb86742786f25817cd0d9070eec8bfc1ee6c5a01c81ff78e247f8dc392"
    ],
    "cHat" : [
      "028627974e31a6ade7c87b6aa0a356a57f7601c8b95b1dc853a488aa2388478e07",
      "02fd1e12d597c8878c7a4322d600028bd5561a43e53c5ae60e78b2f74b0f36afe9",
      "038f0309e14e046dfc4fa8d6cec5e2670c3d49f596fd1b76985e4f098365d1bccd"
    ]
  }
}
```


## E.8 Sub-component pre-decryption-guard

Verification module (checking correctness of operations)

### E.8.1 Verification Component pre-decryption-guard-Polyas

*Computed by authority Polyas*

**Sub-component:** pre-decryption-guard

**Board name:** pre-decryption-guard-Polyas

**Blocked boards (components which wait for this verifier)**:

- decryption-decrypt-Polyas

**Verified components:**

- ballot-box
- ballot-filtered-out
- ballot-flagged
- init-trigger
- keygen-electionKey-Polyas
- mixing-input-packets
- mixing-mix-Polyas
- registry
- revocations

# F Data Types

## F.1 Annotated<B>

Type parameters: `B`

*Description*: A ballot with an additional annotation classifying the ballot as correct, incorrect, or filtered out (revoked)

*Content:*

- `annotation` : `String`
  (*mandatory*)
  If the ballot is not flagged as correct (that is if `status` is not `OK`), this field provides an additional information
- `ballot` : `B`
  (*mandatory*)
  Ballot entry (without annotations)
- `status` : `Status`
  (*mandatory*)
  Status annotation (correct, incorrect, or filtered out)

## F.2 Ballot<GroupElem>

Type parameters: `GroupElem`

*Description*: Encrypted ballot containing (encrypted) voter's choice and appropriate zero-knowledge proofs

*Content:*

- `encryptedChoice` : `MultiCiphertext<GroupElem>`
  (*mandatory*)
  Encrypted voter's choice (represented as a multi-ciphertext)
- `publicLabel` : `String`
  (*mandatory*)
  The public label of the voter who created this ballot
- `reference` : `String`
  (*mandatory*)
  Anonymous voter's reference
- `signature` : `DlogNIZKP.Proof`
  (*mandatory*)
  Schnorr signature on the ballot, created with the voter's private credential
- `zkp` : `List<DlogNIZKP.Proof>`
  (*mandatory*)
  Sequence of zero-knowledge proofs of knowledge of the random coins used to encrypt the voter's choice (one for each ciphertext in the multi-ciphertext `encryptedChoice`)

## F.3 BallotEncodingSpec

*Description*: Type of the encoding used for the ballot, possibly with additional parameters

Sealed class with the following subclasses:

- `Custom`
- `Sparse`
- `Standard`

## F.4 BallotEpEntry<GroupElement>

Type parameters: `GroupElement`

*Description*: Ballot entry containing an encrypted ballot

*Content:*

- `ballot` : `Ballot<GroupElement>`
  (*mandatory*)
  Encrypted ballot, including the voter's public label and reference

## F.5 BallotEpPackage<GroupElement>

Type parameters: `GroupElement`

*Description*: Ballot box entry containing a list of individual (encrypted) ballots

*Content:*

- `ballots` : `List<BallotEpEntry<GroupElement»`
  (*mandatory*)

## F.6 BallotId

*Description*: Ballot identifier (represented as a string)

*Content:*

- `value` : `String`
  (*mandatory*)

## F.7 BallotSpec

*Description*: Specification of a standard ballot content and its validation rules

*Content:*

- `calculateAvailableVotes` : `Boolean`
  (*deprecated, mandatory*)
  If true, the voter front-end application displays the number of available votes. Does not affect validation and counting.
- `contentAbove` : `Content<T>`
  (*optional*)
  Optional content to be displayed at the top of the ballot
- `contentBelow` : `Content<T>`

(*optional*)
Optional content to be displayed at the bottom of the ballot

- `encoding` : `BallotEncodingSpec`
(*mandatory*)
The binary encoding specification of cast ballots
- `externalIdentification` : `String`
(*optional*)
Optional external identification of the ballot supplied by third party vendors like UniWahl4.
- `extraBallotRules` : `Any`
(*optional*)
Additional ballot validation and counting rules, to support forward compatibility
- `id` : `BallotId`
(*mandatory*)
Identifier of the ballot, unique in the scope of the electionTemplate
- `lists` : `List<CandidateListSpec>`
(*mandatory*)
Sequence of candidate lists
- `maxListsWithChoices` : `Int`
(*optional*)
The maximum number of lists where the voter can place his/her choices. If `null`, there are no restrictions.
- `maxVotes` : `Int`
(*mandatory*)
The maximal total number of votes (crosses) of all kinds a voter can select in the whole ballot; ballots with bigger number of crosses will be rejected as invalid
- `maxVotesForCandidates` : `Int`
(*optional*)
The maximal allowed number of votes for candidates (as opposed to votes for lists) in the whole ballot
- `maxVotesForLists` : `Int`
(*optional*)
The maximal allowed number of votes for lists in the whole ballot
- `minVotes` : `Int`
(*mandatory*)
The lower bound on the total number of votes a voter can select in the whole ballot; ballots with smaller number of crosses will be rejected as invalid
- `minVotesForCandidates` : `Int`

(*optional*)
The minimal required number of votes for candidates (as opposed to votes for lists) in the whole ballot

- `minVotesForLists` : `Int`
(*optional*)
The minimal required number of votes for lists in the whole ballot
- `prohibitLessVotes` : `Boolean`
(*deprecated, mandatory*)
If true, the client can't cast a vote with less crosses than required (by minVotes, minVotesForLists, minVotesForCandidates)
- `prohibitMoreVotes` : `Boolean`
(*deprecated, mandatory*)
If true, the client can't cast a vote with more crosses than allowed (by maxVotes, maxVotesForLists, maxVotesForCandidates)
- `showAbstainOption` : `Boolean`
(*mandatory*)
If true, an abstention checkbox is shown
- `showInvalidOption` : `Boolean`
(*mandatory*)
If true, a checkbox is shown for explicitly marking the ballot as invalid
- `title` : `I18n<String>`
(*mandatory*)
Title of the ballot
- `type` : `String`
(*mandatory*)
- `voterClientSettings` : `Any`
(*optional*)
Additional settings of the voter's client (front-end) for configuring the voter's experience

## F.8 CandidateListSpec

*Description*: Specification of a candidate list (a list of candidates/voting options)

*Content:*

- `autofillConfig` : `Any`
(*optional*)

If present autofill is activate, with the provided configuration. This flag only has an effect on the behavior of the voter front-end, not on the validation and counting.

- `candidates` : `List<CandidateSpec>`
  (*mandatory*)
  List of candidate specifications
- `columnHeaders` : `List<I18n<String»`
  (*mandatory*)
  List of column headers
- `columnProperties` : `List<ColumnProperties>`
  (*optional*)
  Special properties for each column
- `contentAbove` : `Content<T>`
  (*optional*)
  Optional content to be displayed above the title
- `derivedCandidateVotes` : `DerivedCandidateVotesSpec`
  (*optional*)
  Optional specification of derived candidate votes, where votes for candidates are derived from votes for the list
- `derivedListVotes` : `DerivedListVotesSpec`
  (*optional*)
  Optional specification of derived list votes, where votes for candidates count as votes for the list
- `externalIdentification` : `String`
  (*optional*)
  Optional external identification of the list supplied by third party vendors like UniWahl4.
- `extraListRules` : `Any`
  (*optional*)
  Additional validation and counting rules, to support forward compatibility
- `id` : `String`
  (*mandatory*)
  The identifier of the list, unique in the scope of the electionTemplate
- `maxVotesForList` : `Int`
  (*mandatory*)
  The maximal allowed number of votes for this list (crosses next to the list header). Non-zero value is used for elections with list voting.
- `maxVotesOnList` : `Int`

(*mandatory*)
The maximal allowed total number of votes (for regular candidates) on the list

- `maxVotesTotal` : `Int`
(*optional*)
The maximal allowed total number of votes (of both types: on and for the list)
- `minVotesForList` : `Int`
(*mandatory*)
The minimal required number of votes for this list (crosses next to the list header)
- `minVotesOnList` : `Int`
(*mandatory*)
The minimal required total number of votes (for regular candidates) on the list
- `minVotesTotal` : `Int`
(*optional*)
The minimal required total number of votes (of both types: on and for the list)
- `title` : `I18n<String>`
(*optional*)
The title (headline) of the candidate list (optional)
- `voteCandidateXorList` : `Boolean`
(*mandatory*)
If true, the voter is not allowed to place votes on the list and for the list at the same time
- `voterClientSettings` : `Any`
(*optional*)
Additional settings of the voter's client (front-end) for configuring the voter's experience

## F.9 CandidateSpec

*Description*: Specification of a candidate (voting option)

*Content:*

- `columns` : `List<Content<T»`
(*mandatory*)
The content of the consecutive columns describing the candidate; the number of columns should correspond to the number of the column headers in the enclosing `CandidateList`
- `externalIdentification` : `String`

(*optional*)
Optional external identification of the candidate supplied by third party vendors like UniWahl4.

- `extraCandidateRules` : `Any`
(*optional*)
Additional validation and counting rules, to support forward compatibility
- `id` : `String`
(*mandatory*)
The identifier of the candidate, unique in the scope of electionTemplate
- `maxVotes` : `Int`
(*mandatory*)
The maximal number of votes that can be given to the candidate
- `minVotes` : `Int`
(*mandatory*)
The minimal required number of votes that must be given to the candidate
- `voterClientSettings` : `Any`
(*optional*)
Additional settings of the voter's client (front-end) for configuring the voter's experience
- `writeInSize` : `Int`
(*optional*)
If non-null and non-zero, the field represents a write-in with the given size, where the content is used as a placeholder

## F.10 Ciphertext<GroupElement>

Type parameters: `GroupElement`

*Description*: ElGamal ciphertext represented as two group elements x and y

*Content:*

- `x` : `GroupElement`
(*mandatory*)
- `y` : `GroupElement`
(*mandatory*)

### F.11 ColumnProperties

*Description*: Special properties for a column in a candidate list

*Content:*

- `hide` : `Boolean`
  (*mandatory*)
  Hide this column in the voter front-end

### F.12 Content<T>

*Description*: Content with a value of some type `T` with internationalization.

Sealed class with the following subclasses:

- `RichText`
- `Text`

### F.13 Custom

*Content:*

- `encodedBallotSize` : `Int`
  (*mandatory*)
- `type` : `String`
  (*mandatory*)
- `jsonTree` : `JsonNode`
  (*mandatory*)

### F.14 DecryptionZKP.Proof<GroupElem>

Type parameters: `GroupElem`

*Description*: A non-interactive zero-knowledge proof of correct decryption

*Content:*

- `decryptionShare` : `GroupElem`
  (*mandatory*)

- `eqlogZKP` : `EqlogNIZKP.Proof`
  (*mandatory*)
  Non-interactive zero-knowledge proof of equality of discrete logarithms

## F.15 DerivedCandidateVotesSpec

*Content:*

- `excludeSelectedCandidates` : `Boolean`
  (*mandatory*)
  If true, the candidates with explicit votes given by the voter are will not obtain additional derived votes
- `variant` : `DerivedCandidateVotesSpec.Variant`
  (*mandatory*)
  The variant of the vote derivation method, specifying how many derived votes a single candidate can obtain

## F.16 DerivedCandidateVotesSpec.Variant

*Description*: The variant of the vote derivation method, specifying how many derived votes a single candidate can obtain

*Enumeration type with values*

- `AT_MOST_ONE`
- `AS_MUCH_AS_POSSIBLE`

## F.17 DerivedListVotesSpec

*Content:*

- `variant` : `DerivedListVotesSpec.Variant`
  (*mandatory*)

## F.18 DerivedListVotesSpec.Variant

*Enumeration type with values*

- `EACH_VOTE_COUNTS`
- `AT_MOST_ONE`

## F.19 DlogNIZKP.Proof

*Description*: Zero-knowledge proof of the knowledge of discrete logarithms.

*Content:*

- `c` : `BigInteger`
  (*mandatory*)
- `f` : `BigInteger`
  (*mandatory*)

## F.20 ECElement

*Description*: A point of the elliptic curve SecP256K1

## F.21 EpRegistry<GroupElement>

Type parameters: `GroupElement`

*Description*: Registry entry containing, in particular, the ballot description and the list of registered voters

*Content:*

- `ballotSigningKey` : `String`
  (*mandatory*)
  Verification key (hex) of the election controller for the signatures on ballot acknowledgements
- `ballotStructures` : `List<BallotSpec>`
  (*mandatory*)
  Description of the ballots
- `electionId` : `String`
  (*mandatory*)
  Unique identifier of the election
- `electionTitle` : `String`
  (*mandatory*)

Description of the election

- `packetSize` : `Int`
  (*mandatory*)
  The maximal size of the mix packet
- `voterIdentifiers` : `List<PublicVoterId>`
  (*mandatory*)
  List of public (non-anonymized) voter IDs along with their corresponding public labels
- `voters` : `List<EpVoter<GroupElement»`
  (*mandatory*)
  List of voters with voter information

### F.22 EpVoter<GroupElement>

Type parameters: `GroupElement`

*Description*: Information about an eligible voter

*Content:*

- `aux` : `Map<String, String>`
  (*optional*)
  Auxiliary data of the voter (e.g. weight or tags), attached to the voter's ballot during tallying
- `cred` : `GroupElement`
  (*mandatory*)
  The public credential of the voter
- `id` : `String`
  (*mandatory*)
  Voter's anonymous unique identifier (the reference)
- `publicLabel` : `String`
  (*mandatory*)
  The public label (":" separated list of ballot sheet identifiers) assigned to the voter

### F.23 EqlogNIZKP.Proof

*Description*: Non-interactive zero-knowledge proof of equality of discrete logarithms

*Content:*

- `c` : `BigInteger`
  (*mandatory*)
- `f` : `BigInteger`
  (*mandatory*)

### F.24 I18n<T>

Type parameters: `T`

*Description*: A value of type `T` with internationalization (possibly different values for different languages)

*Content:*

- `default` : `T`
  (*mandatory*)
  The default value (used when there is no values specified for the requested language)
- `value` : `Map<Language, T>`
  (*mandatory*)
  A mapping from languages to values of type `T`

### F.25 Language

*Content:*

- `languageCode` : `String`
  (*mandatory*)

### F.26 Message

*Description*: Sequence of bytes represented as a hexadecimal string

### F.27 MessageWithProof<G>

Type parameters: `G`

*Description*: A decrypted message along with zero-knowledge proofs of correct decryption

*Content:*

- `message` : `Message`
  (*mandatory*)
  A decrypted message
- `proof` : `List<DecryptionZKP.Proof<G»`
  (*mandatory*)
  Zero-knowledge proofs of correct decryption (one for each ciphertext in the decrypted multi-ciphertext)

### F.28 MessageWithProofPacket<GroupElem>

Type parameters: `GroupElem`

*Description*: A packet of decrypted messages with zero-knowledge proofs of correct decryption

*Content:*

- `auxData` : `Map<String, String>`
  (*optional*)
  The auxiliary data (e.g. weight or tags) shared by all the messages in the packet
- `messagesWithZKP` : `List<MessageWithProof<GroupElem»`
  (*mandatory*)
  List of decrypted messages with zero-knowledge proofs of correct decryption
- `publicLabel` : `String`
  (*mandatory*)
  The public label shared by all the messages in the packet

### F.29 MixPacket<GroupElem>

Type parameters: `GroupElem`

*Description*: Mix packet containing a list of ElGamal multi-ciphertexts plus, optionally, a zero-knowledge proof of correct shuffle

*Content:*

- `auxData` : `Map<String, String>`
  (*optional*)
  The auxiliary data (e.g. weight or tags) shared by all the ciphertexts in the packet
- `ciphertexts` : `List<MultiCiphertext<GroupElem»`
  (*mandatory*)
  A list of ElGamal multi-ciphertexts (encrypted choices)
- `proof` : `ShuffleZKProof<GroupElem>`
  (*optional*)
  A zero-knowledge proof of correct shuffle w.r.t. the preceding mix packet (null for the input packets, before the first shuffle)
- `publicLabel` : `String`
  (*mandatory*)
  The public label shared by all the ciphertexts in the packet

## F.30 MultiCiphertext<GroupElement>

Type parameters: `GroupElement`

*Description*: List of ElGamal ciphertexts (over the given set of group elements); used to represent an encryption of a message which does not fit into one ciphertext

*Content:*

- `ciphertexts` : `List<Ciphertext<GroupElement»`
  (*mandatory*)

## F.31 Packet<A>

Type parameters: `A`

*Content:*

- `values` : `List<A>`
  (*mandatory*)

## F.32 PublicKeyWithZKP<GroupElem>

Type parameters: `GroupElem`

*Description*: Public election key along with a zero-knowledge proof of knowledge of the corresponding private key

*Content:*

- `publicKey` : `GroupElem`
  (*mandatory*)
  The public key
- `zkp` : `DlogNIZKP.Proof`
  (*mandatory*)
  A zero-knowledge proof of the knowledge of the private key corresponding to the public key (a proof of the knowledge of the discrete logarithm)

### F.33 PublicVoterId

*Content:*

- `publicId` : `String`
  (*mandatory*)
- `publicLabel` : `String`
  (*mandatory*)

### F.34 RevocationToken

*Description*: A revocation token specifying voters whose ballots are to be revoked

*Content:*

- `electionId` : `String`
  (*mandatory*)
  Election identifier (hash) specifying the election to which the revocation token applies
- `voterIds` : `List<String>`
  (*mandatory*)
  A list of voters whose ballots are to be revoked

### F.35 RichText

*Content:*

- `value` : `I18n<RichTextDocument>`
  (*mandatory*)
  A value of type `T` with internationalization (possibly different values for different languages)
- `contentType` : `Type`
  (*mandatory*)

## F.36 RichTextDocument

*Content:*

- `data` : `Map<String, String>`
  (*optional*)
- `nodes` : `List<RichTextDocument>`
  (*optional*)
- `text` : `String`
  (*optional*)
- `type` : `String`
  (*optional*)
- `jsonTree` : `JsonNode`
  (*mandatory*)

## F.37 ShuffleZKProof<GroupElement>

Type parameters: `GroupElement`

*Description*: Non-interactive zero-knowledge proof of correct shuffle (Wikstroem et al.)

*Content:*

- `c` : `List<GroupElement>`
  (*mandatory*)
- `cHat` : `List<GroupElement>`
  (*mandatory*)
- `s` : `ZKPs`
  (*mandatory*)
  The s-components of a zero-knowledge proof of correct shuffle

- `t` : `ZKPt<GroupElement>`
  (*mandatory*)
  The t-components of a zero-knowledge proof of correct shuffle

## F.38 SigningKey

*Content:*

- `bytes` : `ByteArray`
  (*mandatory*)

## F.39 SigningKeyPair

*Content:*

- `signingKey` : `SigningKey`
  (*mandatory*)
- `verificationKey` : `VerificationKey`
  (*mandatory*)

## F.40 Sparse

*Content:*

- `bytesPerChoiceId` : `Int`
  (*mandatory*)
  Number of bytes needed to encode one (each) choice index
- `encodedBallotSize` : `Int`
  (*mandatory*)
  The expected size of the encoded ballot in bytes
- `type` : `String`
  (*mandatory*)

## F.41 Standard

*Content:*

- `type` : `String`
  (*mandatory*)

## F.42 Status

*Enumeration type with values*

- `OK`
- `INCORRECT`
- `REVOKED`

## F.43 Text

*Content:*

- `value` : `I18n<String>`
  (*mandatory*)
  A value of type `T` with internationalization (possibly different values for different languages)
- `contentType` : `Type`
  (*mandatory*)

## F.44 Type

*Enumeration type with values*

- `TEXT`
- `RICH_TEXT`

## F.45 VerificationKey

## F.46 ZKPs

*Description*: The s-components of a zero-knowledge proof of correct shuffle

*Content:*

- `s1` : `BigInteger`

(*mandatory*)

- `s2` : `BigInteger`
  (*mandatory*)
- `s3` : `BigInteger`
  (*mandatory*)
- `s4` : `List<BigInteger>`
  (*mandatory*)
- `sHat` : `List<BigInteger>`
  (*mandatory*)
- `sPrime` : `List<BigInteger>`
  (*mandatory*)

### F.47 ZKPt<GroupElement>

Type parameters: `GroupElement`

*Description*: The t-components of a zero-knowledge proof of correct shuffle

*Content:*

- `t1` : `GroupElement`
  (*mandatory*)
- `t2` : `GroupElement`
  (*mandatory*)
- `t3` : `GroupElement`
  (*mandatory*)
- `t4x` : `List<GroupElement>`
  (*mandatory*)
- `t4y` : `List<GroupElement>`
  (*mandatory*)
- `tHat` : `List<GroupElement>`
  (*mandatory*)

## References

[1] Standards for Efficient Cryptography 2 (SEC 2), 2010. Available at http://www.secg.org/sec2-v2.pdf.

[2] David Bernhard, Olivier Pereira, and Bogdan Warinschi. How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings*, volume 7658 of *Lecture Notes in Computer Science*, pages 626–643. Springer, 2012.

[3] David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *Advances in Cryptology - CRYPTO '92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings*, volume 740, pages 89–105. Springer, 1992.

[4] Rolf Haenni, Philipp Locher, Reto E. Koenig, and Eric Dubuis. Pseudo-code algorithms for verifiable re-encryption mix-nets. In *Financial Cryptography and Data Security - FC 2017 International Workshops, WAHC, BITCOIN, VOTING, WTSC, and TA, Sliema, Malta, April 7, 2017, Revised Selected Papers*, volume 10323 of *Lecture Notes in Computer Science*, pages 370–384. Springer, 2017.

[5] Thomas Haines. A Description and Proof of a Generalised and Optimised Variant of Wikström's Mixnet. Technical Report arXiv:1901.08371, arXiv, 2009. Available at https://arxiv.org/abs/1901.08371.

[6] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*, 48(177):203–209, 1987.

[7] Ralf Küsters, Tomasz Truderung, and Andreas Vogt. Clash Attacks on the Verifiability of E-Voting Systems. In *33rd IEEE Symposium on Security and Privacy (S&P 2012)*, pages 395–409. IEEE Computer Society, 2012.

[8] Johannes Müller and Tomasz Truderung. A Protocol for Cast-as-Intended Verifiability with a Second Device. *CoRR*, abs/2304.09456, 2023.

[9] Johannes Müller and Tomasz Truderung. CAISED: A Protocol for Cast-as-Intended Verifiability with a Second Device. In Melanie Volkamer, David Duenas-Cid, Peter Rønne, Peter Y. A. Ryan, Jurlind Budurushi, Oksana Kulyk, Adrià Rodriguez Pérez, and Iuliia Spycher-Krivonosova, editors, *Electronic Voting*, pages 123–139, Cham, 2023. Springer Nature Switzerland.

[10] Claus-Peter Schnorr. Efficient signature generation by smart cards. *J. Cryptology*, 4(3):161–174, 1991.

[11] Björn Terelius and Douglas Wikström. Proofs of Restricted Shuffles. In Daniel J. Bernstein and Tanja Lange, editors, *Progress in Cryptology - AFRICACRYPT 2010, Third International Conference on Cryptology in Africa*, volume 6055 of *Lecture Notes in Computer Science*, pages 100–113. Springer, 2010.

[12] Douglas Wikström. A commitment-consistent proof of a shuffle. *IACR Cryptology ePrint Archive*, 2011:168, 2011.

[13] Douglas Wikström. User Manual for the Verificatum Mix-Net Version 1.4.0. Verificatum AB, Stockholm, Sweden, 2013.